

RaBitQ-SSD: Split Codes and Pipelined I/O for SSD-Resident Vector Search

Yuxuan Xu
Nanyang Technological
University, Singapore
yuxuan001@e.ntu.edu.sg

Jianyang Gao
ETH Zurich
Zurich, Switzerland
jianyang.gao@inf.ethz.ch

Michael Norris
Meta
Menlo Park, United States
mnorris@meta.com

Alibek Zhakubayev
Meta
Menlo Park, United States
alibek@meta.com

Pankaj Singh
Meta
Bellevue, United States
pankajsingh@meta.com

Junjie Qi
Meta
Bellevue, United States
jqj@meta.com

Matthijs Douze
Meta
Paris, France
matthijs@meta.com

Cheng Long
Nanyang Technological
University, Singapore
c.long@ntu.edu.sg

Abstract

SSD-resident approximate nearest-neighbor search is essential when vector collections exceed DRAM capacity. The challenge is to reduce SSD reads and hide I/O latency through concurrent reads and overlap with computation. However, for graph-based search, progressive candidate discovery limits advance I/O planning while IVF search reads inverted lists in full, incurring unnecessary SSD reads. In this paper, we present RaBitQ-SSD, an extension of IVF-RaBitQ for SSD-resident vector search. Specifically, we propose Split-RaBitQ, which retains a configurable prefix of each binary code in DRAM, allowing in-memory code storage below one bit per dimension. Using this partial representation, it provides an unbiased distance estimator with a probabilistic error bound. Once the in-memory coarse quantizer identifies candidate lists, this bound supports pruning individual candidates within them, enabling finer-grained SSD access. We also design an asynchronous search pipeline that coordinates candidate pruning with SSD read scheduling to reduce unnecessary reads while overlapping I/O with computation. On datasets ranging from 5 million to 1 billion vectors, RaBitQ-SSD delivers up to 1.74 $\times$ the throughput of graph-based baselines at 90% recall, while reducing SSD page reads by up to 3.8 $\times$. Its on-SSD indexes are up to 7.0 $\times$ smaller and 10.0 $\times$ faster to build than DiskANN. We also build and search an index of 10 billion vectors on a single machine with one SSD, achieving over 3,000 queries per second at 90% recall.

1 Introduction

Modern embedding models represent unstructured data, such as images and text, as high-dimensional vectors, typically with hundreds to thousands of dimensions. Approximate nearest-neighbor (ANN) search retrieves vectors close to a query without exhaustively scanning the collection and underlies applications such as semantic search, recommendation, and retrieval-augmented generation [32, 43]. As collections grow, however, the vectors themselves can exceed the memory capacity of a single machine: one billion 1024-dimensional vectors in FP32 already occupy 4 TB. Flash storage provides a natural storage tier for such collections: modern flash-based SSDs offer roughly 10–50 $\times$ lower cost per byte than DRAM and a single PCIe 4.0 NVMe SSD can deliver over one million random-read IOPS [25]. Consequently, storing the vectors on SSD while keeping compressed representations in memory has

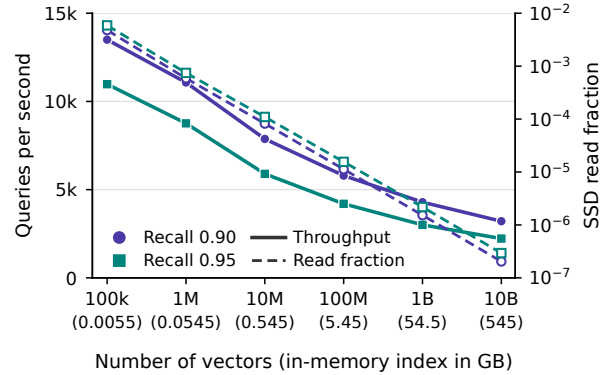


Figure 1: RaBitQ-SSD scales from 100 thousand to 10 billion DINO vectors (1,024 dimensions) on one server with one SSD. Solid lines show throughput for a given recall target (left axis); dashed lines show the fraction of the on-SSD data that is read to perform the search (right axis, log scale): at 10B scale it is less than one millionth.

become a practical and cost-effective design for billion-scale ANN search [9, 22, 53, 63].

DiskANN [53] established a widely adopted graph-based design for SSD-based ANN search in which compact quantized codes in DRAM guide the search while the graph and full-precision vectors reside on SSD. Its synchronous beam search batches reads to exploit device parallelism, but each search round waits for those reads to complete before the next round, limiting I/O-computation overlap. Starling [63] reduces SSD I/O through locality-aware graph reordering and shorter traversal paths. However, as vector dimensionality grows and vertex records become larger, fewer related vertices can be co-located within the same SSD block. PipeANN [22] breaks the strict compute-I/O ordering of conventional beam search by speculatively issuing reads while computation proceeds. However, later I/O completions may reveal better candidates, making some speculative reads unnecessary. Subsequent work further optimizes disk layouts and caching [30, 34, 42, 75], the placement of compressed codes [44, 54, 73], distance computation [10], and I/O-computation scheduling [31]. Despite these optimizations, graph methods with SSD-resident adjacency lists retain a structural constraint: candidates are discovered progressively as the traversal proceeds. Expanding an uncached vertex requires accessing its on-disk record

to obtain its neighbors for subsequent exploration. This causality limits how far SSD accesses can be planned ahead of time.

On the other hand, IVF-based indexes organize vectors into inverted lists, avoiding the cost of building and storing a proximity graph over the full collection. An in-memory coarse index identifies candidate lists before accessing their SSD-resident contents. Among these methods, SPANN [9] uses query-aware pruning to select inverted lists, while HELMSMAN [27] learns how many lists to probe. Both read the selected lists in full from SSD.

This motivates finer-grained I/O pruning within probed inverted lists. If an in-memory representation provides distance estimates with lower bounds for each vector, the bounds can help rule out vectors that cannot improve the current k NN result before fetching their SSD-resident data. The in-memory IVF-RaBitQ [16, 17, 59] supports such error-bounded pruning by maintaining two representations of each vector. It scans a compact D -bit RaBitQ code to estimate distances and prune candidates, and accesses a higher-precision representation only for the surviving candidates, where D denotes the vector dimensionality. This separation naturally suggests an SSD-based extension: retain the compact codes in DRAM for pruning and place the higher-precision representations on SSD for re-ranking, but this exposes two challenges. First, this direct extension retains the full binary code and cannot accommodate memory budgets below one bit per dimension that is necessary on billion scale datasets, e.g., for 10 billion 1024-dimensional vectors, the binary codes alone require about 1.28TB of DRAM. Second, the in-memory pruning needs careful coordination with I/O pipelining so as to reduce unnecessary reads and enable I/O-computation overlap, e.g., fetching the candidates surviving the pruning individually wastes page bandwidth, issuing pages too early can read data that later pruning would eliminate, etc.

In this paper, we present RaBitQ-SSD, an SSD-based extension of in-memory IVF-RaBitQ that addresses these two challenges. First, we propose Split-RaBitQ (§3), which keeps only a *prefix* of $D_m \leq D$ bits of each binary code in memory. Therefore, the memory consumption is configurable by setting D_m . We derive an unbiased distance estimator and a corresponding error bound from the prefix alone, enabling error-bounded pruning. Second, we develop an asynchronous SSD search pipeline that reduces SSD reads while overlapping I/O with computation. The pipeline coalesces surviving candidates at page granularity and issues page reads asynchronously under a bounded in-flight I/O budget. It prioritizes more promising pages and re-evaluates them against the latest distance threshold before issuance, allowing updated search results to eliminate many unnecessary SSD reads without sacrificing I/O concurrency.

In summary, we make the following contributions.

- We propose Split-RaBitQ (§3), which keeps a configurable prefix of each RaBitQ binary code in DRAM. We derive an unbiased distance estimator and a probabilistic error bound from the prefix, enabling candidate pruning with an in-memory code footprint below one bit per dimension.
- We design an asynchronous search pipeline (§4) that reduces unnecessary SSD I/O while sustaining I/O-computation overlap.
- We conduct an extensive evaluation of RaBitQ-SSD (§5) on six datasets ranging from 5M to 1B vectors and 512–1536

dimensions, comparing against strong baselines. We further evaluate scalability from 100 thousand to 10 billion DINO vectors on a single server with one SSD (Figure 1).

RaBitQ-SSD is open-sourced at <https://github.com/VectorDB-NTU/RaBitQ-SSD>.

2 Preliminaries

2.1 ANN Search and IVF

Let $\mathcal{X} \subset \mathbb{R}^D$ be a collection of N data vectors, and let $q_r \in \mathbb{R}^D$ be a query vector. Given a distance function, a k -nearest-neighbor (k NN) query asks for the k vectors in $\mathcal{X}$ closest to q_r . In large-scale vector search, this query is typically answered approximately and thus is called approximate nearest neighbor (ANN) query: an ANN algorithm returns a set $\mathcal{R}$ of k vectors, whose accuracy is measured by $\text{recall}@k = |\mathcal{R} \cap \mathcal{G}|/k$, where $\mathcal{G}$ denotes the exact k NN set.

The inverted file (IVF) [28] is a classical partitioning-based ANN index that underlies many widely deployed vector search systems [11, 61, 65]. At build time, the dataset is partitioned into n_{list} clusters, typically by k -means. The centroids $c_1, \dots, c_{n_{\text{list}}}$ serve as a coarse quantizer, each vector being assigned to its nearest centroid, and the vectors assigned to c_i form the inverted list L_i . At query time, IVF scans the n_{probe} lists whose centroids are closest to q_r , in ascending order of centroid distance; the parameter n_{probe} controls the accuracy–efficiency trade-off. Though IVF narrows the search to the probed lists, scanning a list still requires estimating the distance from q_r to every vector in it, so fast distance estimation with a small memory footprint is crucial. It can be achieved through vector quantization such as RaBitQ [16, 17], which we review next.

2.2 RaBitQ and Its Error Bound

RaBitQ [17] quantizes a vector relative to a center vector, for which the cluster centroid is the natural choice. RaBitQ’s notation is used throughout: for a raw data vector o_r assigned to centroid c and a raw query q_r , $o := (o_r - c)/\|o_r - c\|$ and $q := (q_r - c)/\|q_r - c\|$ are the unit residuals. The distance metric is $d(q_r, o_r) = \|o_r - q_r\|^2$. Expanding either around c yields $d(q_r, o_r) = -2\|o_r - c\|\|q_r - c\|\langle o, q \rangle + K$. K groups the terms precomputed per vector or computed once per probed cluster [17]. Therefore the per-vector computation reduces to estimating the inner product between two unit vectors $\langle o, q \rangle$, and RaBitQ focuses on quantizing unit vectors. This decomposition works similarly when the nearest neighbor search in L2 space is replaced by maximum inner product search.

The codebook of RaBitQ is the set of 2^D vectors $\{\pm 1/\sqrt{D}\}^D$ rotated by a random orthogonal matrix P , sampled once per index. The quantized vector of o is the nearest codeword $\bar{o} := P\bar{x}$, $\bar{x} \in \{\pm 1/\sqrt{D}\}^D$; since $\langle o, P\bar{x} \rangle = \langle P^{-1}o, \bar{x} \rangle$, $\bar{x}$ takes the signs of the coordinates of the rotated vector $P^{-1}o$, stored as the D -bit *binary code* $\bar{x}_b \in \{0, 1\}^D$ with $\bar{x} = (2\bar{x}_b - 1)/\sqrt{D}$.

RaBitQ estimates $\beta = \langle o, q \rangle \approx \hat{\beta}_D = \langle \bar{o}, q \rangle / \langle \bar{o}, o \rangle$, where $\langle \bar{o}, o \rangle$ is a per-vector scalar precomputed at indexing time as follows [17].

$$\langle \bar{o}, o \rangle = \langle \bar{x}, P^{-1}o \rangle = \frac{1}{\sqrt{D}} \sum_{i=1}^D |(P^{-1}o)[i]| \quad (1)$$

LEMMA 2.1 (THEOREM 3.2 OF [17]). $\widehat{\beta}_D = \langle \bar{o}, q \rangle / \langle \bar{o}, o \rangle$ is an unbiased estimator of $\beta = \langle o, q \rangle$, and

$$\Pr \left\{ \left| \beta - \widehat{\beta}_D \right| > \sqrt{\frac{1 - \langle \bar{o}, o \rangle^2}{\langle \bar{o}, o \rangle^2}} \cdot \frac{\epsilon_0}{\sqrt{D-1}} \right\} \leq 2e^{-c_0 \epsilon_0^2},$$

where ϵ_0 is a parameter that controls the failure probability and c_0 is a constant.

Lemma 2.1 gives a confidence interval for $\langle o, q \rangle$: the estimate plus or minus the error bound above. Since the coefficient of $\langle o, q \rangle$ in $d(q_r, o_r)$ is negative, substituting the upper endpoint of its confidence interval yields a lower bound $\text{LB}(q_r, o_r)$ on the distance that holds with probability at least $1 - 2e^{-c_0 \epsilon_0^2}$. The in-memory IVF-RaBitQ uses this bound to prune candidates before re-ranking (§2.3).

The numerator of the estimate $\langle \bar{o}, q \rangle = \langle \bar{x}, P^{-1}q \rangle$ is a sum of D coordinate-wise products that can be stored in a lookup table built once per query [17, 59]. The vectors are laid out in batches of 32 packed codes so that the lookups can be performed from SIMD registers, a technique called *FastScan* [4]. All other terms of the estimate and the bound that depend on the data vector are folded at indexing time into three per-vector scalars, the *factors*, stored beside the code [59]. For higher accuracy, Extended RaBitQ [16] spends B bits per coordinate: the most significant bit is the sign bit above, and the remaining $B - 1$ bits constitute the *ex-code*; its estimate needs two further factors per vector [16, 59]. The multi-bit RaBitQ distance estimation [16] reuses the sign-bit sum and refines it with the ex-code part.

2.3 In-Memory IVF-RaBitQ and a Naive SSD Extension

The in-memory IVF-RaBitQ [16, 17, 59] integrates Extended RaBitQ into an in-memory IVF index. Each inverted list stores the binary codes of its vectors packed in batches of 32 for FastScan. The search maintains KNNSet , the k candidates with the smallest refined distances found so far. The threshold dist_k is the largest of these distances, or $+\infty$ before k candidates have been collected. In a probed list, every batch yields estimates and lower bounds (for batches of 32 candidates). Candidates with $\text{LB}(q_r, o_r) < \text{dist}_k$ are re-ranked using their ex-codes and may update KNNSet ; all others are pruned. No raw vectors are kept; the refined distances are used for the final ranking [16].

The most direct way to port this index to SSD, suggested in [16], keeps the binary code and the three factors in memory, and moves the ex-codes, with their two re-ranking factors, to SSD. The search is unchanged up to re-ranking: when a candidate survives pruning, the page holding its ex-code is read from SSD, scanning blocks until the read completes, and the candidate is re-ranked. However, this naive extension has two problems.

Problem 1: a rigid memory footprint. The binary code has a fixed size, one bit per coordinate, so the in-memory part of the index occupies $ND/8$ bytes plus the factors, which is over 1TB when N is 10 billion and $D = 1024$. In this paper, we present Split-RaBitQ (§3), which keeps only a *prefix* of the binary code in memory, and show that the prefix still yields a lower bound for pruning. As a

result, the size of the in-memory code is configurable by setting the length of the prefix.

Problem 2: blocking I/O. A random SSD read of 4 KB typically takes 10s of microseconds. Issuing one read at a time therefore completes $\sim 10\text{k}$ of reads per second, far below the $1\text{M} + 4\text{KB}$ random reads per second that an SSD can sustain [25]. Reaching this throughput requires hundreds of concurrent outstanding requests. Instead of blocking on every read in the naive extension, we develop an asynchronous pipeline (§4) that overlaps I/O with the computation of scanning the codes in memory.

3 Split-RaBitQ

Split-RaBitQ resolves Problem 1 by splitting the binary code between memory and SSD: §3.1 gives the design, and §3.2 derives the lower bound and its guarantee from RaBitQ’s analysis.

3.1 The Code Splitting Design

Split-RaBitQ keeps RaBitQ’s quantization, and what changes is how many bits of the binary code are kept in memory. The first D_m bits of the binary code with $D_m \leq D$, referred to as the *prefix*, stay in memory together with the three factors used to compute the lower bound in the pruning stage. D_m is fixed at build time according to the RAM budget, and the prefix is packed in batches of 32 for FastScan (§2.2). The remaining $D_s := D - D_m$ bits of the binary code, the ex-code, and the two re-ranking factors reside on SSD.

For codeword $\bar{x}$, let $\bar{x}_{D_m}$ be $\bar{x}$ with the remaining $D - D_m$ coordinates set to zero. Let $\bar{o}_{D_m} := P\bar{x}_{D_m}$. Since P is orthogonal and $\bar{x} \in \{\pm 1/\sqrt{D}\}^D$, we have $\|\bar{o}_{D_m}\|^2 = \|\bar{x}_{D_m}\|^2 = D_m/D$. Split-RaBitQ estimates $\langle o, q \rangle$ from the prefix by

$$\widehat{\beta}_{D_m} = \frac{D}{D_m} \cdot \frac{\langle \bar{o}_{D_m}, q \rangle}{\langle \bar{o}, o \rangle}. \quad (2)$$

As before, $\langle \bar{o}, o \rangle$ is computed at indexing time when all coordinates are available, see Eq. (1). The numerator $\langle \bar{o}_{D_m}, q \rangle = \langle \bar{x}_{D_m}, P^{-1}q \rangle$ is the inner product $\langle \bar{o}, q \rangle$ of §2.2 restricted to the first D_m coordinates. The factor D/D_m compensates for the omitted $D - D_m$ coordinates (§3.2). As in RaBitQ, the estimate and lower bound use three precomputed per-vector factors. For re-ranking, it fetches the remaining $D - D_m$ bits of the binary code, the ex-code, and the factors from SSD and computes the estimate based on the codes as in the original RaBitQ method (§2.2).

3.2 A Lower Bound from the Prefix Code

Let $\beta := \langle o, q \rangle$. The projection of q onto o is βo , and the remaining component $q - \beta o$ is perpendicular to o . For non-collinear o and q , let e_1 be the unit vector along this perpendicular component. Then $q = \beta o + \sqrt{1 - \beta^2} e_1$. For collinear o and q , the perpendicular component vanishes. Substituting this decomposition into Eq. (2), the Split-RaBitQ estimation error becomes

$$\widehat{\beta}_{D_m} - \beta = \beta \left(\frac{D}{D_m} \frac{\langle \bar{o}_{D_m}, o \rangle}{\langle \bar{o}, o \rangle} - 1 \right) + \sqrt{1 - \beta^2} \frac{D}{D_m} \frac{\langle \bar{o}_{D_m}, e_1 \rangle}{\langle \bar{o}, o \rangle}. \quad (3)$$

For the full code ($D_m = D$), the first term vanishes and the remaining term is RaBitQ’s estimation error.

By Eq. (1), $\langle \bar{o}_{D_m}, o \rangle / \langle \bar{o}, o \rangle$ is the fraction of the ℓ_1 mass of $P^{-1}o$ that lies in the prefix coordinates. By exchangeability of the coordinates of $P^{-1}o$, each coordinate contributes a $1/D$ share of this mass

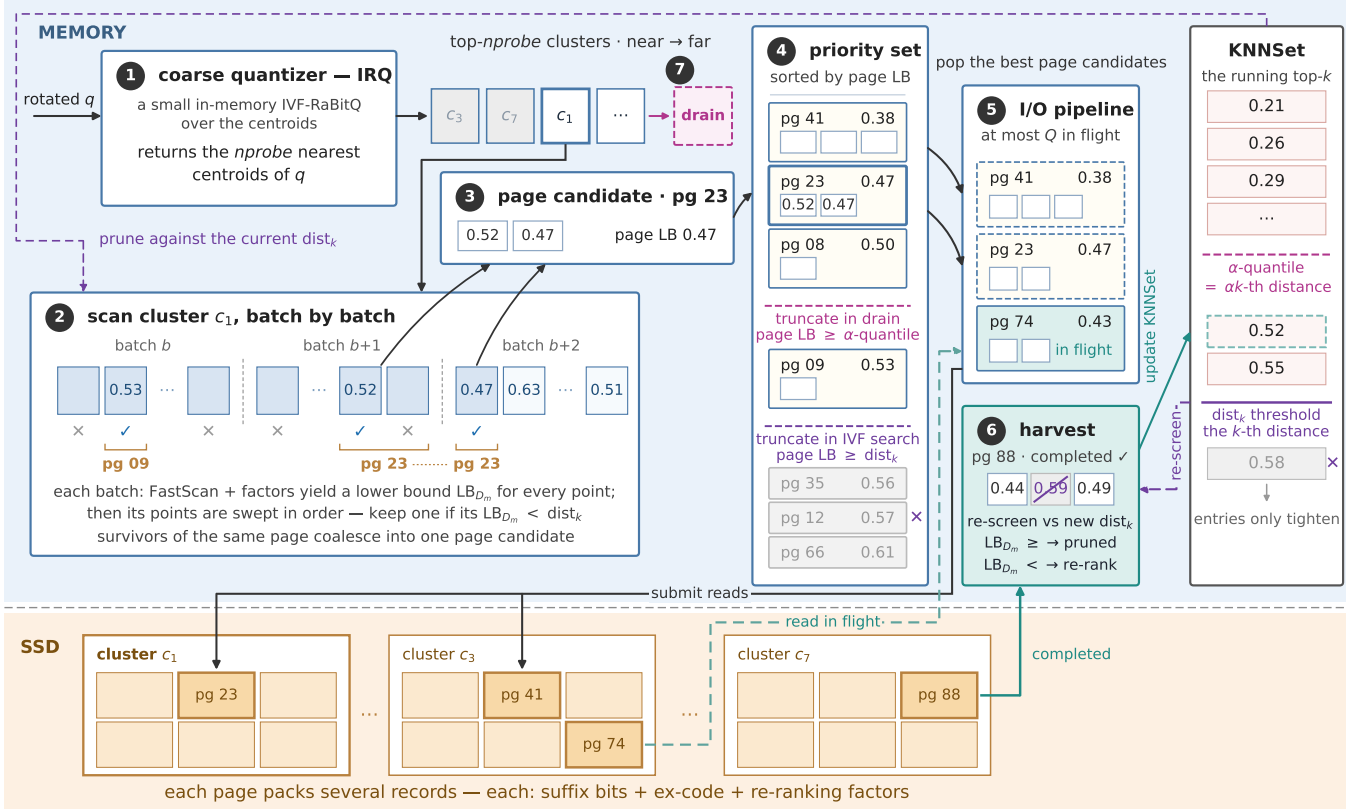


Figure 2: Overview of the RaBitQ-SSD search pipeline. Memory (top) holds the coarse quantizer and the per-vector prefix codes; the SSD (bottom) holds one record per vector, in cluster-contiguous 4 KB pages. See § 4.1 for details on the search process.

in expectation. The scaling factor D/D_m rescales the prefix estimate to the full vector. The deviation of the scaled prefix fraction from one produces the first error term, while the second comes from the component of $\hat{o}_{D_m}$ along e_1 . Theorem 3.1 states the error bound.

THEOREM 3.1 (PREFIX ESTIMATOR). $\hat{\beta}_{D_m} = \frac{D}{D_m} \frac{\langle \hat{o}_{D_m}, q \rangle}{\langle \hat{o}, o \rangle}$ is an unbiased estimator of $\langle o, q \rangle$. For $0 < \epsilon \leq \sqrt{D_m}/3$, the error bound of the estimator is given as

$$\Pr \left\{ \left| \hat{\beta}_{D_m} - \langle o, q \rangle \right| > \frac{\epsilon}{\sqrt{D_m}} \right\} \leq 2e^{-c_1 \epsilon^2},$$

where c_1 is a constant.

The proof is given in Appendix A. As in RaBitQ, the error bound can be concisely presented as an estimation error of $O(1/\sqrt{D_m})$ with high probability.

Lower bound. Substituting the estimate $\hat{\beta}_{D_m} + \epsilon/\sqrt{D_m}$ for $\langle o, q \rangle$ in $d(q_r, o_r)$ gives the lower bound $LB_{D_m}(q_r, o_r)$, which lies below the true distance with high probability $1 - 2e^{-c_1 \epsilon^2}$. Appendix A.4 examines the empirical error distribution and the frequency of lower-bound violations.

Reducing D_m saves memory and computation during the pruning phase but widens the error bound, potentially increasing SSD reads and re-ranking work. We evaluate this trade-off in §5.3.

4 Search Pipeline

4.1 Overview

Figure 2 gives an overview of the search pipeline.

Storage. The in-memory coarse quantizer routes a query to its n_{probe} nearest clusters. Each cluster stores, for every vector, the D_m -bit prefix of its binary code together with its three factors (§3.1), stored in batches of size 32. The SSD holds one fixed-size record per vector, consisting of the suffix bits of the binary code, the ex-code, and the two re-ranking factors. The records of a cluster are packed into contiguous 4 kB pages (§2.3), with some padding so that they don't cross page boundaries. The in-RAM and SSD vectors are stored in the same order (by ascending distance to the centroid), so that the page containing a record can be determined directly.

Searching. A KNNSet keeps track of the nearest k vectors seen so far and their distances to the query, whose maximum is $dist_k$. One query proceeds as follows. ① The rotated query passes the coarse quantizer, an in-memory IVF-RaBitQ (IRQ) over the cluster centroids (§4.2), which returns the n_{probe} nearest clusters, visited from near to far. ② Within a cluster, the scan proceeds by FastScan batches of 32 vectors. For each vector, the prefix codes yield the lower bound and a point “survives” if it is below the current $dist_k$. ③ Surviving points of the same page coalesce into a page candidate, ④ which enters the priority set keyed by the page lower bound, the minimum over its members. ⑤ Reads are issued asynchronously

from the priority set while the scan continues. After each batch, the search polls for completions and issues additional reads while keeping at most Q reads in flight per query. Before issuing a read, the search re-checks the page candidate against the current dist_k . ⑥ A completed page is harvested: members that fail the current dist_k are dropped, and the rest are re-ranked from the fetched data and may update the KNNSet. The details of pipelining the inverted-list search will be presented in §4.3. ⑦ After the last cluster, the search drains the set and the reads in flight, popping only candidates whose bound falls below the tighter α -quantile threshold (§4.4).

4.2 IRQ: In-Memory IVF-RaBitQ as the Coarse Quantizer

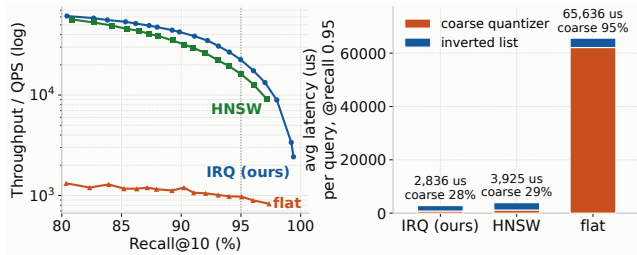


Figure 3: Coarse quantizers on YFCC-100M ($C = 96,422$, $k = 10$, $T = 64$). Left: throughput against recall. Right: average query latency at recall 0.95, split into the coarse step and the rest (“inverted list”).

Before any inverted list is visited, the coarse quantizer finds the n_{probe} nearest of the $C = n_{\text{list}}$ centroids. A flat scan spends C full-precision distance computations on this regardless of n_{probe} , while the work that follows scales with n_{probe} . On YFCC-100M ($C = 96,422$, some 10^3 points per cluster), the flat scan accounts for 95% of the query latency at recall@10 = 0.95: 62 of the 66 ms spent on a query go to routing alone (Fig. 3, right). Building a graph index over the centroids is the standard remedy [6, 11]: with HNSW, average query latency falls to 3.9 ms, of which the coarse step is 29%.

We adopt another coarse quantizer, an in-memory IVF-RaBitQ built over the centroids, *IRQ* for short. Centroids are quantized to $B = 9$ bits per dimension by default, which is sufficient for high accuracy [16], and a query retrieves its n_{probe} nearest centroids through the prune-and-re-rank flow of §2.3. IRQ reduces the average query latency to 2.8 ms (23× below the flat scan, better than the graph index; Fig. 3, right) and achieves the best end-to-end performance (Fig. 3, left). Storing the centroids as B -bit codes also keeps the coarse quantizer smaller.

4.3 Pipelining the Inverted-List Search

RaBitQ-SSD scans the n_{probe} inverted lists, for which we design several pipelines of increasing complexity, starting from the naive extension of §2.3. See Table 1 for the experimental setup and I/O numbers cited below.

Naive. The naive extension of §2.3 issues one read per surviving candidate and blocks on it. It has one virtue: every pruning decision

sees the tightest dist_k available so far, because it is fully causal. The query thread, however, waits out every read: a query issues 102 page reads at $k=10$ and 614 at $k=100$ on average, each delivering one useful record out of the three a page holds, and the thread waits on I/O for 95–99% of the list-search time (Table 1).

Coalesced. Points of the same page that survive pruning can share one read (③ in Fig. 2): because the scan visits the points of a page consecutively (§4.1), the surviving points of a page emerge consecutively, and its read is issued when the scan crosses the page boundary. The speedup is due to fewer page reads: for $k = 10$, 102 reads per query become 81, or 1.30× throughput (1.66× throughput for $k=100$). The tightest-threshold property of Naive is preserved at page granularity.

Overlapped. Blocking is what stalls the query thread (the grey strips of Naive and Coalesced in Fig. 4), and this step removes it. The scan no longer waits for reads: the surviving points of a page form a *page candidate*, whose read is submitted when all points of a page have been scanned. After each batch, the search polls for completed reads without blocking and *harvests* them (⑥ in Fig. 2): every member is re-screened against the current dist_k , and those that pass are re-ranked. When the last cluster has been scanned, the search keeps harvesting until every outstanding read returns, i.e., the *drain* (⑦ in Fig. 2). As dist_k may have tightened since the read was submitted, some pages may have been fetched for nothing, which Table 1 reports as *waste*. How Overlapped performs depends on whether the SSD is saturated. Below its IOPS ceiling ($T=1$), the overlap is a clear gain: over Coalesced, throughput rises another 5.1× at $k=10$ (9.4× at $k=100$), even though 156 reads are issued where Coalesced needed 81. These 46% wasted reads are acceptable because the benefit of overlapping I/O outweighs the cost of the extra reads. Once the device is saturated ($T=64$, where the 64 threads together drive the SSD at its IOPS ceiling), additional reads begin to queue. I/O completions slow down, delaying both harvesting and tightening of dist_k . The resulting stale threshold allows more candidates to survive pruning and triggers still more reads. In our trace, the number of outstanding reads for one query peaks at 677 in Fig. 4b. In this regime a query wastes 83% of the reads and spends 89% of its list-search time in the drain (Table 1). Throughput falls below Coalesced at both k (0.45× and 0.84×) and, at $k=10$, below even the blocking Naive.

Scheduled. This step bounds the number of reads in flight and moves each read decision from scan time to issue time, when the threshold is tighter. Instead of having its read submitted as soon as the scan leaves the page, a page candidate is deposited in the *priority set* (④ in Fig. 2), ordered by page lower bound. Reads are issued in that order, keeping at most Q in flight (⑤). After each batch, the search polls and harvests as before, and conditionally submits new reads by order of lower bound: (a) if fewer than Q are in flight and (b) if each candidate lower bound is lower than the current dist_k . The issuance stops at the first candidate that fails check (b) as the remaining candidates would fail too or the set is emptied. While a candidate waits in the set the threshold keeps tightening, and most candidates are pruned there without a read (*drop* column of Table 1). Because Scheduled keeps fewer reads in flight than Overlapped, its threshold is slightly more stalled and more points survive the scan, but many page candidates are still dropped before incurring I/Os. Fewer fetched pages also mean less harvest work, visible in Fig. 4b

Table 1: Page reads for five I/O pipeline designs of RaBitQ-SSD, on YFCC-100M at recall 0.90 (per-query means). QPS× is relative to Naive at the same T and k . *issued*: 4 KB page reads submitted. *drop*: page candidates discarded in the pending set before any read. *waste*: issued reads whose every member the re-screen pruned (zero re-rank). *re-rank/surv.*: records re-ranked over points surviving the scan. *io-wait*: share of the list-search wall blocked on reads (blocking designs). *drain*: share of the same wall spent finishing outstanding I/O after the scan (async designs). [†]See §4.4.

design	$T = 1$ thread							$T = 64$ threads						
	QPS×	issued	drop	waste	re-rank/surv.	io-wait	drain	QPS×	issued	drop	waste	re-rank/surv.	io-wait	drain
$k = 10$														
Naive	1.00	102	—	—	102/102	95%	—	1.00	102	—	—	102/102	93%	—
Coalesced	1.30	81	—	—	102/103	94%	—	1.30	81	—	—	102/103	96%	—
Overlapped	6.67	156	—	72	104/334	—	4%	0.58	483	—	400	103/833	—	89%
Scheduled	8.35	53	797	8	61/2292	—	6%	6.98	40	1204	1	52/3414	—	21%
Full	8.36	54	779	8	61/2247	—	5%	7.32	38	1078 [†]	1	50/3056 [†]	—	10%
$k = 100$														
Naive	1.00	614	—	—	614/614	99%	—	1.00	614	—	—	614/614	99%	—
Coalesced	1.66	439	—	—	614/615	98%	—	1.41	439	—	—	614/615	98%	—
Overlapped	15.62	482	—	39	623/750	—	2%	1.18	1382	—	941	615/2773	—	66%
Scheduled	19.94	241	1432	1	363/4367	—	13%	6.89	236	6042	0	357/17040	—	73%
Full	21.89	212	1495	0	333/4457	—	5%	12.35	131	5076 [†]	0	248/13738 [†]	—	19%

as the shorter blue bar of Scheduled: Overlapped fetches 483 pages to Scheduled’s 40 (103 vs. 52 re-ranks).

Discussion. Figure 4 shows the SSD reads in flight and CPU activity for each of the pipelines, depending on the number of threads T . Below the IOPS ceiling ($T=1$), the gain over Overlapped is modest, 1.25× at $k=10$ and 1.28× at $k=100$, because wasted reads add little wall time when the device is not saturated. The number of reads, however, falls from 156 to 53 per query, fewer than even the blocking Coalesced (81).

Once the device is saturated ($T=64$), the advantage is decisive: 40 reads per query, essentially none wasted, and the in-flight count stays flat at Q (8 here, 16 at $T=1$) instead of rising into the hundreds (Fig. 4); the traces show 27.9 ms for Overlapped and 1.5 ms for Scheduled, and throughput is 12× Overlapped at $k=10$ and 5.8× at $k=100$ (Table 1).

We further evaluate the effect of lower-bound ordering by replacing it with FIFO while keeping all other settings of Scheduled unchanged. At $T = 64$ and recall 0.90, FIFO issues roughly twice as many reads per query, reducing throughput to 43% and 53% of Scheduled’s for $k = 10$ and $k = 100$, respectively.

4.4 α -Quantile Threshold to Alleviate Drain

A drainage problem. The drain runs after the last cluster, as no scanning is left to run in parallel with reads. Below the IOPS ceiling ($T=1$) reads complete promptly, little remains after the scan, and the drain represents 6–13% of the list-search time (Table 1). Once the device is saturated ($T=64$), completions slow down (in our traces the median read completes in 186 μ s), so fewer reads are issued during the scan under the bound Q , more page candidates are left when it ends, and the drain takes 21% of the list-search time at $k=10$ and even 73% at $k=100$.

Full. This pipeline issues fewer reads by tightening the drain’s threshold. Once the last cluster is scanned (Ⓒ in Fig. 2), a pending candidate is issued only if it could still place among the best αk of

the current results, with $\alpha \in (0, 1]$ and αk an integer. This means its lower bound falls below the α -quantile of the KNNSet. $\alpha = 1$ recovers dist_k and hence Scheduled, and $\alpha < 1$ gives a tighter threshold. The tighter threshold applies only to reading decisions; harvested pages are still re-screened against dist_k .

The threshold comes with a guarantee: with exact re-ranking and valid lower bounds, every point among the true top- αk keeps its lower bound below the α -quantile threshold and is never pruned. In this way, the search guarantees a recall of at least α over the points the probed clusters contain. In practice a far larger fraction of the true top- k survives the α -quantile. We measured this in the tightest possible setting on YFCC-100M, with the running KNNSet replaced by the true top- k so that the threshold is the αk -th smallest true distance: at $\alpha = 0.4$, about 85% of the true top- k neighbors have a lower bound below this threshold and would not be pruned (84.2% at $k=10$, 85.1% at $k=100$), against the 40% the guarantee promises, and we adopt 0.4 as the default.

Impact on recall and determinism. In the actual search, the recall loss incurred by the α -quantile threshold is even smaller than suggested by this measurement, for three reasons: (1) the running KNNSet typically gives a looser threshold than the true top- k ; (2) only the candidates still pending when the drain begins face the α -quantile threshold; and (3) pruning is per page candidate, against the smallest lower bound among its members, so one strong surviving point keeps its whole page.

We note that, with exact re-ranking and valid lower bounds, I/O completion order does not affect the top- k set returned by Scheduled ($\alpha = 1$), but can affect that returned by Full ($\alpha < 1$). In Full, depending on the order reads finish, re-ranking these candidates can tighten the α -quantile threshold sooner or later. Thus, I/O completion order can affect which neighbors are retained at ranks $(\alpha k, k]$. Appendix B shows that the effect of I/O completion order on the returned neighbor sets and recall is low.

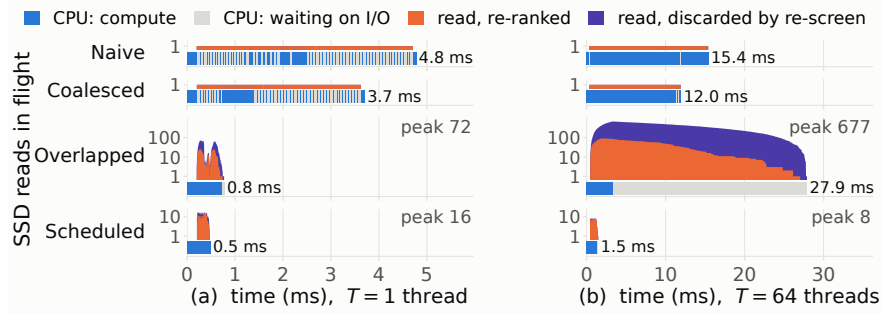


Figure 4: Measured timeline of one query under each pipeline, at (a) $T=1$ and (b) $T=64$ (YFCC-100M, $k=10$, recall 0.90; the median-latency query of a full 10k-query run). Per pipelining step, the lower strip shows the CPU state over time, compute (blue) or waiting on I/O (grey), and the area above counts SSD reads in flight (log scale), split into reads whose page fed a re-rank (orange) and reads discarded whole by the re-screen (violet). Annotations give the peak in-flight count and the total latency.

The gain follows the drain. Below the IOPS ceiling ($T=1$), where the drain is light, Full matches Scheduled at $k=10$ (1.00 $\times$) and gains 1.10 $\times$ at $k=100$. Once the device is saturated ($T=64$), Full reads 131 pages per query at $k=100$ where Scheduled reads 236, the drain share falls from 73% to 19%, and throughput rises 1.8 $\times$. As every query’s drain issues fewer reads, contention on the SSD decreases, allowing scan-phase reads to complete sooner and the scan threshold to tighten earlier, so fewer points survive the scan (the daggered counts of surviving points in Table 1: 13,738 versus 17,040 at $k=100$), which leaves fewer pending candidates to drain.

Table 2: Datasets. All vectors are float32 except those in DINO-10B, which are uint8; the columns give the modality, the dimensionality D , base count, the search metric and whether queries are distributed with the dataset.

Dataset	Modality	D	#Base	Metric	queries?
OpenAI-5M [43]	Text	1536	5 M	ℓ_2	no
Wiki-10M [46]	Text	768	10 M	ℓ_2	yes
DPR-100M [1]	Text	768	101.2 M	IP	no
LAION-100M [50]	Image	768	102.1 M	ℓ_2	yes
YFCC-100M [7]	Image	1280	98.7 M	ℓ_2	yes
DataComp-1B [15]	Image	512	1 B	ℓ_2	no
DINO-10B [40]	Image	1024	10 B	ℓ_2	yes

5 Experimental Study

5.1 Experimental Setup

Platform. All experiments run on a server with an AMD Ryzen Threadripper PRO 9975WX (32 cores, 64 threads), 768 GB of DDR5 memory (8×96 GB, 8 channels), and a Samsung PM9D3a 15.36 TB NVMe SSD attached over PCIe 4.0 $\times 4$, under Ubuntu 24.04 (Linux 6.17); all binaries are compiled with GCC 13.3 at `-O3 -march=native`. We measured that this configuration sustains $r_{\max}=1.63$ M random 4 KB reads per second. We use this rate as the device reference in the I/O analysis. All SSD accesses, ours and every baseline’s, go through libaio under `O_DIRECT` [37], bypassing the page cache. Each query vector is run on one thread, and the number $T \leq 64$ of concurrent threads depends on the experiment.

Datasets and queries. Table 2 lists the seven datasets used in our evaluation.¹ The first six datasets are evaluated with 10,000 queries. For datasets that do not provide a dedicated query set we sample queries disjoint from the base. Ground truth is computed by exhaustive search. DINO-10B and its evaluation setup are described separately in §5.4.

Our configuration. Unless stated otherwise, RaBitQ-SSD uses $B = 9$ bits per dimension (i.e., 8-bit ex-codes) and $C = n_{\text{list}} = N/1024$ clusters. Coarse quantization uses IRQ (§4.2), which encodes the C centroids at 9 bits per dimension and organizes them into $C/1000$ inverted lists. It probes $\gamma \cdot n_{\text{probe}}$ of these lists with $\gamma = 0.5$. The drain threshold is $\alpha = 0.4$ (§4.4); the per-thread in-flight I/O budget is $Q = 16$ pages for $T \leq 16$ and $Q = 8$ for $T > 16$ (§4.3). We conduct the parameter study on the in-flight budget Q and the drain threshold α in Appendix C.3.

Baseline methods. In the main evaluation, we compare against three graph-based SSD-resident systems: DiskANN [41, 53], Starling [63, 76], and PipeANN [22, 55]. We follow prior work [10, 22] in configuring these graph baselines and use their default on-disk representations, which store full-precision vectors. Each disk graph is built with maximum out-degree $R = 64$ and construction list size $L = 200$; Starling and PipeANN build their in-memory navigation graphs from 1% data samples; DiskANN’s node cache is provisioned with 1% of the raw dataset size; and the search beam widths are 8, 8, and 32 for DiskANN, Starling, and PipeANN, respectively. We report comparisons with SPANN [9] and AlayaLaser [3, 10] in §5.6, together with their configurations.

Memory budgets. We consider two memory regimes: low memory sets $D_m = D/2$ and high memory is $D_m = D$. We equalize the per-vector in-memory storage across RaBitQ-SSD and the three graph baselines. For each setting of D_m , the per-point PQ bytes of DiskANN, Starling, and PipeANN equal the per-point size of our in-memory prefix code and factors, i.e., $D_m/8 + 12$ bytes. All indexes are built with the same temporary memory budget: 128 GB per dataset, and 512 GB for DataComp-1B.

¹All data access and experiments were performed by Nanyang Technological University.

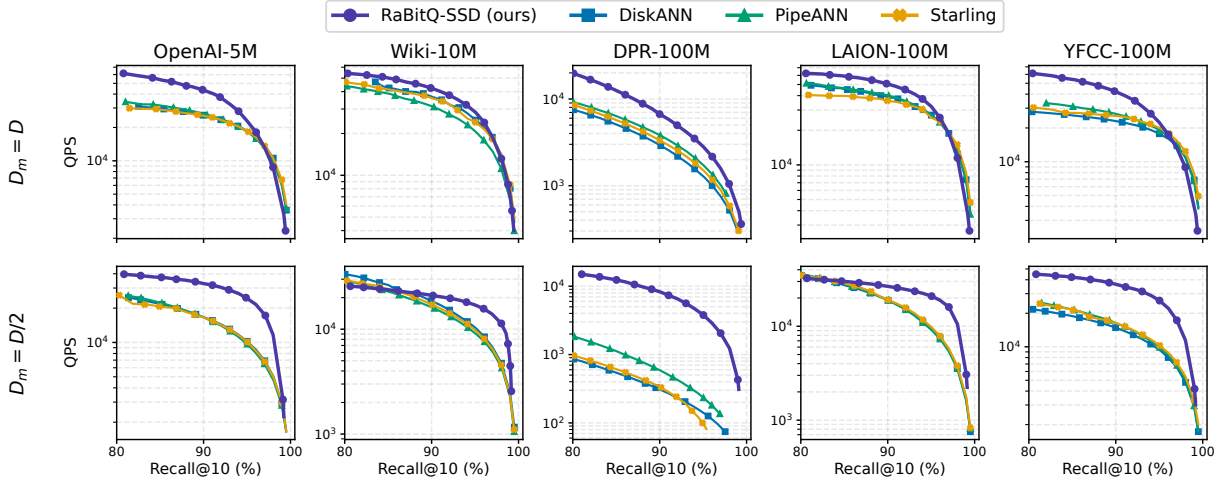


Figure 5: Main evaluation at $k = 10$ on the five datasets (columns): throughput against recall at $T = 64$ (log scale, up is better), under two memory budgets, $D_m = D$ (top row) and $D_m = D/2$ (bottom row), with baselines using matched per-point memory budgets (§5.1).

Metrics. We report **recall@ k** , per-query latency at $T = 1$ and throughput (queries per second, **QPS**) for $T > 1$. When QPS is indicated in plots, it is reported in log scale. To quantify SSD traffic, we also report the mean number of 4 KB **pages read per query**. We aggregate this number into the **read rate** (pages / s), obtained by multiplying the QPS with the average number of page reads per query. The read rate is capped by $r_{\max}$. Curves sweep each method’s accuracy parameter (n_{probe} for RaBitQ-SSD, the search list size for the graph baselines). For comparisons at fixed recall, we pick the smallest setting of n_{probe} or search list size that achieves this recall.

5.2 Main Evaluation

Figure 5 shows that RaBitQ-SSD achieves the highest throughput on all five datasets at recalls 0.90 and 0.95 under both memory budgets. RaBitQ-SSD’s advantage is strongest for the limited memory budget ($D_m = D/2$) at recall 0.95, where the speedup is 1.64–2.52 $\times$ w.r.t. the best baselines, and even 16.18 $\times$ for DPR-100M.

Page reads. Table 3 reports the pages read per query. RaBitQ-SSD reads fewer pages per query than every baseline. The lowest baseline page count is still 1.1–3.8 $\times$ ours. We combine in-memory pruning and page-level I/O scheduling with compact on-disk records, which reduce the number of page reads issued per query and increase the observed throughput. On the four datasets other than DPR-100M, and for all methods, the read rate is 1.4–1.7M/s, close to $r_{\max}$. When the device is saturated, fewer pages per query translate into higher throughput: on YFCC-100M, our 37.8 pages against PipeANN’s 62.2 accompany a 1.58 $\times$ throughput advantage.

High recall operating points. At recall 0.98 and when $D_m = D$, the fastest graph baseline overtakes RaBitQ-SSD on each of the four datasets other than DPR-100M. In this regime, fewer page reads do *not* always yield higher throughput. On Wiki-10M, RaBitQ-SSD *still* reads fewer pages per query than DiskANN (73.2 versus 108.7), and sustains a lower page-read rate (0.97M versus 1.54M per second). However, reaching this recall requires a larger n_{probe} , increasing

Table 3: 4 KB pages read per query at recall 0.90 ($k = 10$, $T = 64$, $D_m = D$). In bold: the lowest (best) per column.

	OpenAI	Wiki	DPR	LAION	YFCC
RaBitQ-SSD	37.5	37.9	112.6	32.3	37.8
DiskANN	64.1	41.2	555.7	40.4	72.3
PipeANN	62.3	51.5	426.5	38.9	62.2
Starling	63.3	43.0	489.3	38.0	63.1

in-memory scanning work, so computation becomes the limiting factor for throughput.

Smaller memory budget. At $D_m = D/2$ and recall 0.98, however, RaBitQ-SSD achieves 2.11–2.80 $\times$ the throughput of the graph baselines on the four datasets other than DPR-100M. Reducing D_m loosens the lower bounds, allowing more candidates to survive pruning. On Wiki-10M, RaBitQ-SSD reads 141.7 pages per query and sustains 1.61M page reads per second, close to the device limit. The increased I/O limits throughput to about 11.4K QPS, below the 13.3K achieved at $D_m = D$. Similarly, for the graph baselines, shorter in-memory PQ codes yield less accurate distance estimates, requiring more graph exploration and page reads to reach the same recall: DiskANN reads 329.3 pages per query and achieves only 4.8K QPS, leaving RaBitQ-SSD with a 2.37 $\times$ advantage.

The DPR-100M outlier. The larger throughput advantage on DPR-100M at the smaller memory budget reflects a particularly large increase in the graph baselines’ I/O cost. At recall 0.90, shortening their in-memory PQ codes from 108 to 60 bytes increases page reads per query by 6.5–9.6 $\times$. Their page-read rates remain close to 1.6M per second, so throughput falls by 6.5–9.9 $\times$. In comparison, RaBitQ-SSD resists better: page reads per query increase by a factor of only 1.73 (from 112.6 to 194.8). The shorter prefix also reduces FastScan work, allowing throughput to increase by 24% despite the additional I/O (§5.3). Consequently, RaBitQ-SSD’s throughput

Table 4: Per-query latency (ms) at $T = 1$: P50/P95/P99 at recall 0.90, $k = 10$, under both memory budgets.

		OpenAI-5M	Wiki-10M	DPR-100M	LAION-100M	YFCC-100M
$D_m = D$	RaBitQ-SSD	0.61 / 0.79 / 0.86	0.64 / 0.80 / 0.87	3.79 / 4.27 / 4.52	0.53 / 0.69 / 0.76	0.63 / 0.79 / 0.87
	DiskANN	0.80 / 1.05 / 1.22	0.81 / 1.09 / 1.26	8.72 / 9.47 / 10.5	0.76 / 1.05 / 1.25	0.87 / 1.22 / 1.42
	PipeANN	0.60 / 0.82 / 0.96	0.64 / 0.95 / 1.15	2.63 / 3.02 / 3.25	0.62 / 0.89 / 1.12	0.67 / 0.94 / 1.11
	Starling	0.91 / 1.22 / 1.42	0.83 / 1.10 / 1.29	7.71 / 8.45 / 9.14	0.82 / 1.13 / 1.42	0.97 / 1.34 / 1.56
$D_m = D/2$	RaBitQ-SSD	0.66 / 0.89 / 1.04	0.76 / 1.01 / 1.27	3.80 / 5.23 / 5.93	0.62 / 1.00 / 1.53	0.68 / 0.90 / 1.00
	DiskANN	0.98 / 1.23 / 1.40	1.40 / 1.67 / 1.87	78.6 / 83.4 / 86.4	1.34 / 1.64 / 1.89	1.09 / 1.43 / 1.64
	PipeANN	0.70 / 0.92 / 1.07	0.85 / 1.12 / 1.32	17.0 / 18.9 / 20.2	0.82 / 1.11 / 1.37	0.78 / 1.06 / 1.25
	Starling	1.12 / 1.42 / 1.62	1.51 / 1.77 / 1.97	97.9 / 103.2 / 105.8	1.39 / 1.74 / 2.05	1.23 / 1.61 / 1.86

advantage over the fastest baseline increases from $1.74\times$ at $D_m = D$ to $14\times$ at $D_m = D/2$.

Latency variance. Tail latency matters at production request volumes, as even a small fraction of slow requests translates to a large absolute number. Table 4 reports P50, P95, and P99 per-query latencies (i.e., the 50%, 95% and 99% latency quantiles) at recall 0.90 with $T = 1$ under both memory budgets. RaBitQ-SSD also achieves the lowest P95 and P99 latencies in most settings.

5.3 Performance under Varying Operating Conditions

Increasing k . Figure 6 extends the throughput comparison to $k = 100$ and $k = 1000$ on all five datasets. RaBitQ-SSD maintains higher throughput than the baselines. At recall 0.90, its advantage over the fastest baseline increases with k on every dataset, reaching $1.31\text{--}3.31\times$ at $k = 100$ and $1.65\text{--}4.01\times$ at $k = 1000$. Raising k increases the computations and SSD page reads required to retrieve more neighbors: for RaBitQ-SSD, the larger distance threshold allows more candidates to survive pruning, and graph baselines need to explore more nodes to find a larger result set. For example, on YFCC-100M at recall 0.90, RaBitQ-SSD reads 37.8, 132, and 957

pages per query at $k = 10, 100$, and 1000 while the fastest baseline at each k reads 62.2, 326, and 2,803. Since in this regime, the SSD throughput saturates, this translates to a larger throughput advantage for RaBitQ-SSD: from $1.58\times$ at $k = 10$ to $2.42\times$ at $k = 100$ and then to $2.86\times$ at $k = 1000$. RaBitQ-SSD benefits from the IVF data layout: the candidates to be fetched are more likely to be co-located in the same SSD page than with graph based methods.

Query concurrency. An ANN service must use its available SSD read capacity efficiently, particularly when it shares the device with other service instances on the same machine. We examine how throughput changes as increasing query concurrency drives the SSD from low utilization toward saturation.

Figure 7(a) reports the throughput against the read rate. At low query concurrency, the achieved read rates are well below $r_{\max}$. More concurrent queries can sustain more outstanding I/O requests, better exploiting the SSD’s internal parallelism and raising both its utilization and query throughput. Once the device is saturated, additional query concurrency yields little further increase in the SSD read rate. Query throughput is then limited by $r_{\max}$ divided by the number of pages read per query. Additional query concurrency can increase I/O queuing delays and hence per-query latency. At

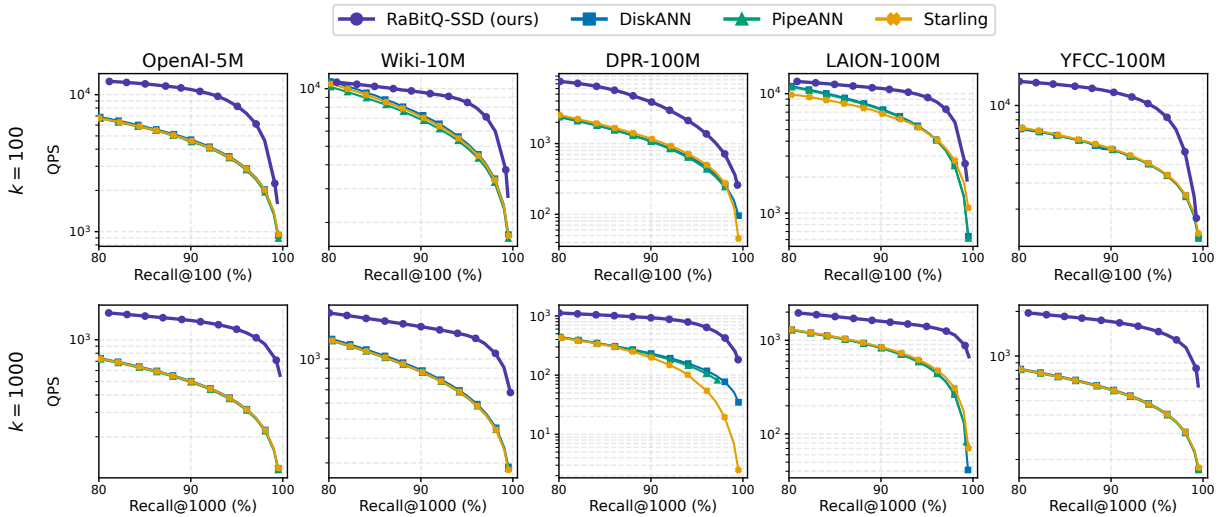


Figure 6: Throughput against recall on the five datasets (columns) at $T = 64$ and $D_m = D$, for $k = 100$ (top row) and $k = 1000$ (bottom row). The QPS axes use a log scale.

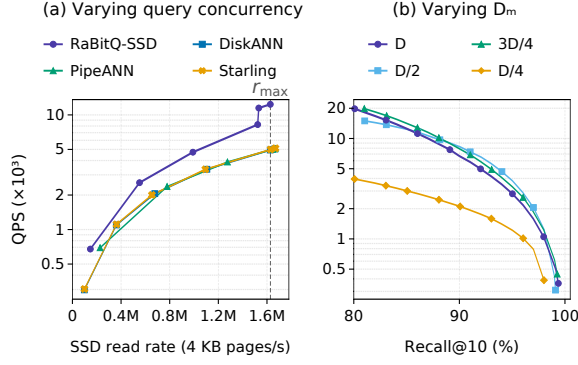


Figure 7: Throughput (QPS, log scale) under varying operating conditions.

(a) Varying query concurrency on YFCC-100M ($D_m = D$, $k = 100$, recall 0.90). Curves are obtained by sweeping $T \in \{1, 4, 8, 16, 32, 64\}$. Vertical line = maximal SSD read rate $r_{\max}$. (b) Varying D_m on DPR-100M ($D = 768$, $k = 10$, $T = 64$)

$T = 64$, all four methods sustain 1.63–1.67M page reads per second, close to $r_{\max}$. RaBitQ-SSD reads about 132 pages per query against the baselines’ 326–329, accounting for its 12,371 QPS against their 5,049–5,118. The graph baselines approach saturation at lower query concurrency, so most points cluster near the right end of the curves. For these baselines, increasing T from 32 to 64 hardly increases throughput, while mean query latency approximately doubles, from 6.3–6.4 ms to 12.4–12.6 ms.

Varying the in-memory size. Figure 7(b) compares Split-RaBitQ with $D_m \in \{D/4, D/2, 3D/4, D\}$. The per-point in-memory footprint is $D_m/8 + 12$ bytes; reducing D_m from D to $D/4$ shrinks it from 108 to 36 bytes (3.0 $\times$), and moves more data to SSD. This cuts the FastScan work per candidate but loosens the bound (width $\propto 1/\sqrt{D_m}$, §3.2), increasing SSD reads and re-ranking work for the same recall.

On DPR-100M at recall 0.90, search at $D_m = D$ is compute-bound, with an SSD read rate of only 0.75M pages per second. Reducing D_m relieves this computational bottleneck. Although the looser bound increases reads per query, the SSD can accommodate the additional I/O, allowing throughput to improve. At $3D/4$, the read rate is still only 1.13M pages per second; at $D/2$, it reaches 1.62M, close to $r_{\max}$. Throughput at $D/2$ reaches 8,298 QPS, 24% above D and the highest among the four settings, despite the increase from 113 pages per query at $D_m = D$ to 195 at $D_m = D/2$. With the SSD near saturation at $D_m = D/2$, further shortening FastScan provides little benefit: reducing D_m to $D/4$ increases the pages to read to 769, so the throughput consequently falls to 2,117 QPS.

5.4 Scaling to Billions of Points

We evaluate RaBitQ-SSD’s behavior closer to the memory and disk capacity of our test machine.

1B scale. We evaluate this scale on DataComp-1B. We set $D_m = D = 512$, so the in-memory codes and factors occupy 76 bytes per point. We compare against DiskANN and PipeANN with per-point PQ size matched at the same 76 bytes. Figure 8 shows the comparison

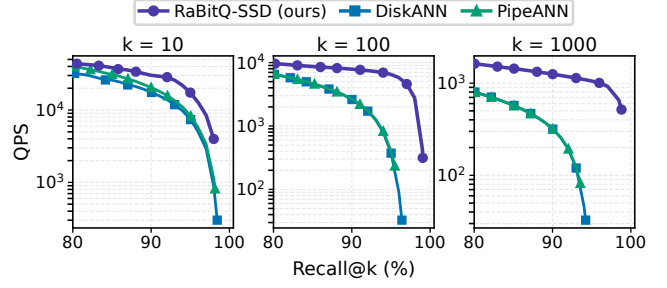


Figure 8: DataComp-1B: throughput against recall at $T = 64$ and $D_m = D = 512$, for varying k .

at $T = 64$. RaBitQ-SSD retains its throughput advantage at this scale. At recall 0.90, its throughput is 1.48 $\times$, 2.96 $\times$, and 3.9 $\times$ that of the fastest baseline for $k = 10, 100$, and 1000, respectively. As in §5.3, the advantage increases with k .

10B scale. DINO-10B [40] holds 10^{10} vectors of 1024 dimensions in uint8 under ℓ_2 , and comes with 100,000 queries and ground truth at $k = 10$, on which we evaluate at $T = 64$. With 8-byte point IDs, the full-prefix codes, factors, and IDs alone require 1.48 TB, exceeding our server’s 768 GB of DRAM. Setting $D_m = 256$ reduces the resident index to 545 GB including the coarse quantizer (with $C \approx N/4096$ clusters). We store the raw uint8 vectors on SSD instead of ex-codes for re-ranking, occupying 10.24 TB. We do not evaluate DiskANN or PipeANN at this scale, as 10B nodes cannot be represented in their 32-bit internal node IDs [41, 55]. An extension to 64-bit IDs, retaining their existing disk layouts, would require an estimated 20.48 TB for the main disk index alone, exceeding our SSD’s capacity. The bulkiness of the edge storage is a fundamental limitation of graph based methods.

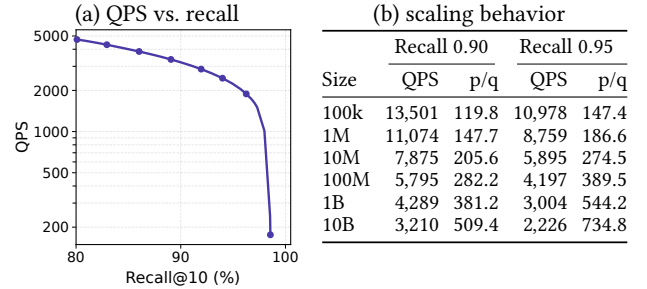


Figure 9: RaBitQ-SSD on DINO-10B, $D_m = D/4 = 256$, $k = 10$, and $T = 64$. (a) throughput against recall (b) throughput and pages per query on subsets of the dataset.

Figure 9(a) shows throughput against recall on DINO-10B. RaBitQ-SSD achieves approximately 3,210 and 2,226 QPS at recall 0.90 and 0.95, respectively. To examine how performance scales with dataset size, we also evaluate subsets containing the first 100k, 1M, 10M, 100M, and 1B vectors, still with $D_m = 256$ and $C \approx N/4096$. Figure 9(b) and Figure 1 summarize the results. As the dataset grows from 100k to 10B points, a 100,000-fold increase in size, throughput decreases by just of 4.21 $\times$ and 4.93 $\times$ at recalls 0.90 and 0.95, respectively.

With $D_m = D/4$, the shorter in-memory code prefix loosens the lower bounds, allowing more candidates to survive pruning and increasing SSD reads. Across all six scales, the achieved read rates are about 1.55–1.64M pages per second, close to $r_{\max}$, indicating I/O-bound search. This is consistent with the tradeoff observed in the varying- D_m experiment (§5.3).

5.5 Ablation Study

The ablation varies three design choices: (1) which code is held in memory, (2) which representation is stored on the SSD, and (3) the precision used for re-ranking. Each variant changes one component while keeping the other index and system settings fixed. Table 5 reports QPS and pages read per query at recall 0.90 ($k = 10$, $T = 64$, $D_m = D$). The results at other recalls are presented in Appendix C.2.

Table 5: Ablation of the in-memory code, on-disk representation, and re-ranking precision, with recall 0.90, $k = 10$, $T = 64$, $D_m = D$; bits/dim = bits per dimension on disk, p/q = pages read per query.

Variant	bits/dim	YFCC-100M		LAION-100M	
		QPS	p/q	QPS	p/q
RaBitQ-SSD ($B = 9$)	8	42,057	37.8	50,509	32.3
RaBitQ-SSD-fp	32	24,952	61.7	43,805	36.8
RaBitQ-SSD (SQ8)	8	41,659	37.5	50,051	32.6
RaBitQ-SSD ($B = 5$)	4	40,688	38.3	48,099	33.2
RaBitQ-SSD (SQ4)	4	30,824	48.3	38,178	42.7
PipeANN (PQ)	32	26,701	62.2	40,233	38.9
PipeANN (RaBitQ)	32	25,591	64.9	38,811	40.4
DiskANN	32	23,017	72.3	39,366	40.4
DiskANN (disk-PQ)	8	37,580	36.5	37,102	41.2

The in-memory code. We evaluate several combinations of in-memory representations for the different methods. PipeANN supports both PQ and RaBitQ codes [55], allowing to compare them directly. Running it with 1-bit RaBitQ codes instead of PQ codes, with its disk graph and layout unchanged, lowers throughput by $\sim 4\%$ on both datasets: merely replacing the in-memory code for navigation offers no throughput benefit. The superior results of RaBitQ-SSD come from the bound provided by Split-RaBitQ (§3.2) to prune candidates.

The on-disk representation. *RaBitQ-SSD-fp* stores the raw fp32 vector in place of our ex-codes, to match the default storage of the baselines, whereas *DiskANN (disk-PQ)* replaces the full-precision vector inside each graph node with a PQ code of one byte per dimension. On YFCC-100M, a compressed on-disk representation reduces the pages needed per record from two to one, lowering page reads per query. This improves throughput by 60+% for both DiskANN and RaBitQ-SSD relative to their fp32 variants, and RaBitQ-SSD still achieves 12% higher throughput than DiskANN (disk-PQ).

On LAION-100M, fp32 records already fit in one page, so this effect does not apply: at recall 0.90, DiskANN (disk-PQ) achieves 6% lower throughput than fp32 DiskANN. This, in turn, stems from the quantization error introduced by disk-PQ, which necessitates

additional search work to reach the same recall. However, in RaBitQ-SSD, compact ex-codes stored contiguously within inverted lists allow more surviving candidates to share a page read, which yields 15% higher throughput over RaBitQ-SSD-fp.

Re-ranking precision. With SQ8 and SQ4, we replace ex-codes with 8-bit or 4-bit scalar quantization with residual encoding [11] on both datasets. At recall 0.90, $B = 9$ and SQ8 have similar throughput, whereas $B = 5$ achieves $1.32\times$ and $1.26\times$ the throughput of SQ4 on YFCC-100M and LAION-100M, respectively. SQ4 requires more probing and page reads to reach the same recall. This shows that RaBitQ-SSD combines the 1-bit RaBitQ code and the 4-bit ex-code to produce a more accurate representation than just 4-bit SQ4.

5.6 Additional baselines

SPANN. SPANN partitions the base into a large number of inverted lists, keeps their centroids in memory, and reads the probed lists from SSD [9]. Figure 10(a) reports the comparison on Wiki-10M. Memory budgets are not matched: SPANN’s in-memory index takes 6.6 GB against our 1.1 GB. At recall 0.90, RaBitQ-SSD delivers $21.3\times$ SPANN’s throughput, because SPANN reads 870 pages per query, compared to ~ 38 for RaBitQ-SSD. SPANN fetches whole selected posting lists, while RaBitQ-SSD performs finer-grained SSD accesses.

AlayaLaser. AlayaLaser [10] is a graph-based index that accelerates distance estimation using PCA and a SIMD-friendly layout that stores neighbors’ quantized codes in each node’s on-disk record. It caches node records in memory. We use the graph parameters specified in §5.1, with beam width 16 and PCA dimension 256. Therefore, for AlayaLaser, there is a throughput against memory usage tradeoff, which Figure 10(b) shows. Under the memory budget matched to our $D_m = D$ configuration, AlayaLaser caches about 2.8% of the nodes. AlayaLaser achieves 8,592 QPS, compared with 28,571 QPS for RaBitQ-SSD. Limited cache coverage leaves many node accesses to SSD, with each cache miss reading a two-page node record. To reach RaBitQ-SSD’s QPS performance, AlayaLaser needs to cache $\sim 50\%$ of the nodes (30 GB of memory), at which point the relevance of a hybrid memory-SSD setting becomes questionable.

5.7 Indexing Cost and Footprints

Table 6 reports the end-to-end build time and the on-SSD index size of RaBitQ-SSD and the three main graph baselines on six datasets, at $D_m = D$. All builds run on CPU only, with 64 threads and the build-memory budget of §5.1.

RaBitQ-SSD builds indexes at least $5.0\times$ faster than the fastest graph baseline on each dataset. On DataComp-1B, it takes 9.2 hours, compared with 45.6 hours for DiskANN. Our build has three stages: training the coarse quantizer, assignment, and encoding. Training runs two-level k -means [13] on a sample of the training vectors; assignment uses HNSW search over the centroids rather than an exhaustive centroid scan for each point. On DataComp-1B, the three stages take 1.8, 5.2, and 2.2 hours, respectively.

RaBitQ-SSD stores compact ex-codes and factors on SSD, whereas the default graph indexes store full-precision vectors and adjacency lists. Using compact codes and omitting per-point adjacency lists reduces our on-SSD index size by a factor of 4.0–7.0 relative to these baselines. On DataComp-1B, RaBitQ-SSD’s on-SSD

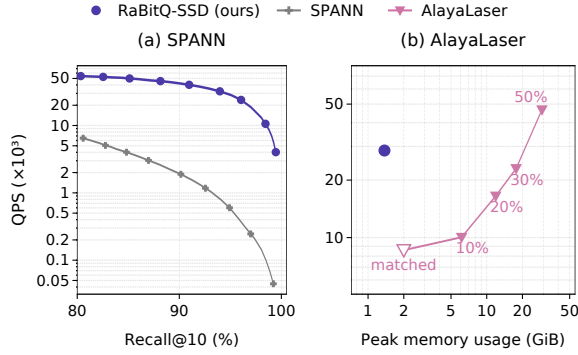


Figure 10: Comparisons on Wiki-10M ($T = 64$, $k = 10$; RaBitQ-SSD at $D_m = D$). (a) Throughput against recall; SPANN uses its default configuration. (b) Throughput against peak memory usage at recall 0.95. “matched” denotes AlayaLaser’s configuration under the matched memory budget; percentages denote cached nodes.

index occupies 587 GB, compared with 4,096 GB for DiskANN and PipeANN. DiskANN’s disk-PQ variant, which stores PQ codes on the SSD in place of the vectors, narrows this storage gap to 1.5–1.7 $\times$ on YFCC-100M and LAION-100M (§5.5).

Table 6: End-to-end build time (hours) / on-SSD index size (GB), at $D_m = D$. Results use each method’s default disk representation. “–”: not built.

dataset	OpenAI	Wiki	DPR	LAION	YFCC	DataComp
N	5M	10M	100M	100M	100M	1B
RaBitQ-SSD	0.05/10.2	0.06/8.2	0.9/83.1	0.8/83.8	1.5/135	9.2/587
DiskANN	0.5/40.9	0.5/41.0	7.9/414	5.5/418	9.1/809	45.6/4,096
PipeANN	0.4/40.9	0.5/41.0	8.8/414	6.3/418	9.3/809	45.9/4,096
Starling	0.9/40.9	1.0/41.0	19.6/414	12.1/418	18.7/809	–

6 Related Work

SSD-resident vector search. Among graph-based methods, DiskANN [53] sets the pattern most later methods follow: compressed codes stay in memory to guide a best-first traversal, while the graph itself resides on SSD. Subsequent graph-based methods reorganize the on-disk layout so that each fetched page serves more of the search [30, 34, 42, 63, 68, 69, 75] and overlap the traversal with asynchronous reads [22, 31]. AlayaLaser accelerates distance computation [10]. A recent study explores this design space [35]. Among partitioning-based methods, SPANN [9] keeps centroids in memory and reads probed lists from SSD; HELMSMAN [27] rebuilds the I/O path on SPDK to saturate its NVMe arrays and uses learned models to choose the probed lists; and OrchANN [8] orchestrates out-of-core search under skewed partitions. Among other methods, LM-DiskANN [44] and AiSAQ [54] move even the compressed codes to the SSD; VeloANN [73] stores RaBitQ ex-codes on the SSD, with its in-memory 1-bit code ordering the graph walk; and SkipDisk [52] prunes reads with a triangle-inequality bound

tightened by per-point pivots. Furthermore, some disk-resident indexes have been extended to support updates [23, 49, 51, 66, 74]. In RaBitQ-SSD, the in-memory prefix of the 1-bit RaBitQ code gives every point its own bound (§3), and only the pages of surviving candidates are read (§4).

Vector quantization. A substantial body of work has investigated the quantization of high-dimensional vectors across multiple fields, including machine learning, computer vision, and data management [2, 5, 14, 16, 17, 19, 20, 24, 28, 29, 38, 45, 56–58, 64, 70, 72]. We refer readers to existing surveys, experimental studies, and textbooks for more comprehensive overviews of related techniques [12, 36, 39, 47, 60, 62, 71]. This paper builds on RaBitQ [17], which provides probabilistic bounds on distance-estimation error, and its extension supporting flexible bit budgets [16] (§2.2). In particular, RaBitQ has been integrated with in-memory graph indexes [21], made GPU-native [48], and run as a heterogeneous NPU–CPU system [26]. Some recent quantization methods are built on top of RaBitQ: MRQ [67] truncates PCA-projected dimensions and quantizes the kept dimensions with RaBitQ; and SAQ [33] allocates bits across segments of PCA-projected dimensions non-uniformly and quantizes each segment with a variant of RaBitQ. Split-RaBitQ instead derives an unbiased distance estimator and a probabilistic error bound from a prefix of the 1-bit RaBitQ code.

7 Conclusion

We presented RaBitQ-SSD, a partitioning-based algorithm for SSD-resident vector search. Its *Split-RaBitQ* design retains a configurable code prefix in memory, enabling fine-grained pruning under different memory budgets. Its asynchronous pipeline overlaps computation with I/O while reducing unnecessary SSD reads. Experiments show that RaBitQ-SSD provides higher throughput, fewer SSD reads, smaller indexes, and faster construction than comparable graph-based systems from the state of the art. It also searches 10 billion vectors using a single server and one NVMe SSD, demonstrating its scalability.

References

- [1] Cecilia Aguerreber, Ishwar Bhati, Mark Hildebrand, Mariano Tepper, and Ted Willke. 2023. Similarity search in the blink of an eye with compressed indices. *Proceedings of the VLDB Endowment* 16, 11 (2023), 3433–3446.
- [2] Cecilia Aguerreber, Ishwar Singh Bhati, Mark Hildebrand, Mariano Tepper, and Theodore Willke. 2023. Similarity Search in the Blink of an Eye with Compressed Indices. *Proc. VLDB Endow.* 16, 11 (July 2023), 3433–3446. <https://doi.org/10.14778/3611479.3611537>
- [3] AlayaDB AI. [n.d.]. AlayaLite. <https://github.com/AlayaDB-AI/AlayaLite>. v1.0.0 branch.
- [4] Fabien André, Anne-Marie Kermarrec, and Nicolas Le Scouarnec. 2015. Cache locality is not enough: high-performance nearest neighbor search with product quantization fast scan. *Proc. VLDB Endow.* 9, 4 (Dec. 2015), 288–299. <https://doi.org/10.14778/2856318.2856324>
- [5] Artem Babenko and Victor Lempitsky. 2014. Additive Quantization for Extreme Vector Compression. In *2014 IEEE Conference on Computer Vision and Pattern Recognition*. 931–938. <https://doi.org/10.1109/CVPR.2014.124>
- [6] Dmitry Baranchuk, Artem Babenko, and Yury Malkov. 2018. Revisiting the inverted indices for billion-scale approximate nearest neighbors. In *Proceedings of the European Conference on Computer Vision (ECCV)*. 202–216.
- [7] Big-ANN Benchmarks. 2025. YFCC-Images Dataset. https://github.com/harshasimhadri/big-ann-benchmarks/blob/main/dataset_preparation/yfccimages_dataset.md. Accessed: 2026-06-08.
- [8] Lizheng Chen, Pinhuan Wang, Shaonan Ma, Jie Zhang, Qing Wang, Zhengyi Yang, Heng Zhang, Mingxing Zhang, Fan Yang, Guihai Chen, Chen Tian, and Chengying Huan. 2026. OrchANN: Hierarchical Orchestration for Skewed Out-of-Core Vector Search. arXiv:2512.22838 [cs.DB] <https://arxiv.org/abs/2512.22838>

- [9] Qi Chen, Bing Zhao, Haidong Wang, Mingqin Li, Chuanjie Liu, Zengzhong Li, Mao Yang, and Jingdong Wang. 2021. SPANN: highly-efficient billion-scale approximate nearest neighbor search. In *Proceedings of the 35th International Conference on Neural Information Processing Systems (NIPS '21)*. Curran Associates Inc., Red Hook, NY, USA, Article 398, 14 pages.
- [10] Weijian Chen, Haotian Liu, Yangsheng Deng, Long Xiang, Liang Huang, and Bo Tang. 2026. AlayaLaser: Efficient Index Layout and Search Strategy for Large-scale High-dimensional Vector Similarity Search. arXiv:2602.23342 [cs.DB] <https://arxiv.org/abs/2602.23342>
- [11] Matthijs Douze, Alexandr Guzhva, Chengqi Deng, Jeff Johnson, Gergely Szilvasy, Pierre-Emmanuel Mazaré, Maria Lomeli, Lucas Hosseini, and Hervé Jégou. 2026. The Faiss Library. *IEEE Transactions on Big Data* 12, 2 (2026), 346–361. <https://doi.org/10.1109/TBDATA.2025.3618474>
- [12] Karima Echihiabi, Kostas Zoumpatianos, Themis Palpanas, and Houada Benbrahim. 2018. The Lernaean Hydra of Data Series Similarity Search: An Experimental Evaluation of the State of the Art. *Proc. VLDB Endow.* 12, 2 (oct 2018), 112–127. <https://doi.org/10.14778/3282495.3282498>
- [13] Faiss wiki. 2026. 2-level clustering. <https://github.com/facebookresearch/faiss/wiki/K%2E2%80%90means-clustering#2-level-clustering>. Accessed: 2026-10-01.
- [14] Hakan Ferhatosmanoglu, Ertem Tuncel, Divyakant Agrawal, and Amr El Abbadi. 2000. Vector Approximation Based Indexing for Non-Uniform High Dimensional Data Sets. In *Proceedings of the Ninth International Conference on Information and Knowledge Management (McLean, Virginia, USA) (CIKM '00)*. Association for Computing Machinery, New York, NY, USA, 202–209. <https://doi.org/10.1145/354756.354820>
- [15] Samir Yitzhak Gadre, Gabriel Ilharco, Alex Fang, Jonathan Hayase, Georgios Smyrnis, Thao Nguyen, Ryan Marten, Mitchell Wortsman, Dhruva Ghosh, Jieyu Zhang, Eyal Orgad, Rahim Entezari, Giannis Daras, Sarah Pratt, Vivek Ramanujan, Yonatan Bitton, Kalyani Marathe, Stephen Mussmann, Richard Vencu, Mehdi Cherti, Ranjay Krishna, Pang Wei Koh, Olga Saukh, Alexander Ratner, Shun Song, Hannaneh Hajishirzi, Ali Farhadi, Romain Beaumont, Sewoong Oh, Alex Dimakis, Jenia Jitsev, Yair Carmon, Vaishal Shankar, and Ludwig Schmidt. 2023. DataComp: In search of the next generation of multimodal datasets. *arXiv preprint arXiv:2304.14108* (2023).
- [16] Jianyang Gao, Yutong Gou, Yuexuan Xu, Yongyi Yang, Cheng Long, and Raymond Chi-Wing Wong. 2025. Practical and Asymptotically Optimal Quantization of High-Dimensional Vectors in Euclidean Space for Approximate Nearest Neighbor Search. *Proc. ACM Manag. Data* 3, 3, Article 202 (June 2025), 26 pages. <https://doi.org/10.1145/3725413>
- [17] Jianyang Gao and Cheng Long. 2024. RaBitQ: Quantizing High-Dimensional Vectors with a Theoretical Error Bound for Approximate Nearest Neighbor Search. *Proc. ACM Manag. Data* 2, 3, Article 167 (May 2024), 27 pages. <https://doi.org/10.1145/3654970>
- [18] Jianyang Gao and Cheng Long. 2024. RaBitQ: Quantizing High-Dimensional Vectors with a Theoretical Error Bound for Approximate Nearest Neighbor Search (Technical Report). https://github.com/gaoj0017/RaBitQ/blob/main/technical_report.pdf.
- [19] Tiezheng Ge, Kaiming He, Qifa Ke, and Jian Sun. 2013. Optimized product quantization for approximate nearest neighbor search. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*. 2946–2953.
- [20] Yunchao Gong, Svetlana Lazebnik, Albert Gordo, and Florent Perronnin. 2013. Iterative Quantization: A Procrustean Approach to Learning Binary Codes for Large-Scale Image Retrieval. *IEEE Transactions on Pattern Analysis and Machine Intelligence* 35, 12 (2013), 2916–2929. <https://doi.org/10.1109/TPAMI.2012.193>
- [21] Yutong Gou, Jianyang Gao, Yuexuan Xu, and Cheng Long. 2025. SymphonyQG: Towards Symphonious Integration of Quantization and Graph for Approximate Nearest Neighbor Search. *Proc. ACM Manag. Data* 3, 1, Article 80 (Feb. 2025), 26 pages. <https://doi.org/10.1145/3709730>
- [22] Hao Guo and Youyou Lu. 2025. Achieving Low-Latency Graph-Based Vector Search via Aligning Best-First Search Algorithm with SSD. In *19th USENIX Symposium on Operating Systems Design and Implementation (OSDI 25)*. USENIX Association, Boston, MA, 171–186. <https://www.usenix.org/conference/osdi25/presentation/guo>
- [23] Hao Guo and Youyou Lu. 2026. OdinANN: Direct Insert for Consistently Stable Performance in Billion-Scale Graph-Based Vector Search. In *24th USENIX Conference on File and Storage Technologies (FAST 26)*. USENIX Association, Santa Clara, CA, 133–147. <https://www.usenix.org/conference/fast26/presentation/guo>
- [24] Ruiqi Guo, Philip Sun, Erik Lindgren, Quan Geng, David Simcha, Felix Chern, and Sanjiv Kumar. 2020. Accelerating large-scale inference with anisotropic vector quantization. In *Proceedings of the 37th International Conference on Machine Learning (ICML '20)*. JMLR.org, Article 364, 10 pages.
- [25] Gabriel Haas and Viktor Leis. 2023. What Modern NVMe Storage Can Do, and How to Exploit it: High-Performance I/O for High-Performance Storage Engines. *Proc. VLDB Endow.* 16, 9 (May 2023), 2090–2102. <https://doi.org/10.14778/3598581.3598584>
- [26] Fujun He, Chuyue Ye, Huaxiang Cai, Zetao Lv, Baolong Cui, Wenru Yan, Chao Zhan, Zigang Zhang, Hao Yi, Jie Xiang, Xiabing Li, Yuhang Gai, Ziyang Zhang, Pengfei Zheng, and Yunfei Du. 2026. Ascend-RaBitQ: Heterogeneous NPU-CPU Acceleration of Billion-Scale Similarity Search with 1-bit Quantization. arXiv:2605.16007 [cs.IR] <https://arxiv.org/abs/2605.16007>
- [27] Yuchen Huang, Baiteng Ma, Yiping Sun, Yang Shi, Xiao Chen, Xiaocheng Zhong, Zhiyong Wang, Yao Hu, Erci Xu, and Chuliang Weng. 2026. The Clustering Strikes Back: Building Cost-Effective and High-Performance ANNS at Scale with Helmsman (Operational Systems). In *20th USENIX Symposium on Operating Systems Design and Implementation (OSDI 26)*. USENIX Association, Seattle, WA, 1623–1639. <https://www.usenix.org/conference/osdi26/presentation/huang-yuchen>
- [28] Hervé Jégou, Matthijs Douze, and Cordelia Schmid. 2011. Product Quantization for Nearest Neighbor Search. *IEEE Trans. Pattern Anal. Mach. Intell.* 33, 1 (Jan. 2011), 117–128. <https://doi.org/10.1109/TPAMI.2010.57>
- [29] Wenqi Jiang, Shigang Li, Yu Zhu, Johannes De Fine Licht, Zhenhao He, Runbin Shi, Cedric Renggli, Shuai Zhang, Theodoros Rekatsinas, Torsten Hoefler, and Gustavo Alonso. 2023. Co-design Hardware and Algorithm for Vector Search. In *Proceedings of the International Conference for High Performance Computing, Networking, Storage and Analysis (Denver, CO, USA) (SC '23)*. Association for Computing Machinery, New York, NY, USA, Article 87, 15 pages. <https://doi.org/10.1145/3581784.3607045>
- [30] Dingyi Kang, Dongming Jiang, Hanshen Yang, Hang Liu, and Bingzhe Li. 2025. Scalable Disk-Based Approximate Nearest Neighbor Search with Page-Aligned Graph. arXiv:2509.25487 [cs.LG] <https://arxiv.org/abs/2509.25487>
- [31] Dingyi Kang, Juncheng Yang, and Bingzhe Li. 2026. LAANN: I/O-Aware Look-Ahead Search for Disk-Based Approximate Nearest Neighbor Search. arXiv:2606.02784 [cs.DB] <https://arxiv.org/abs/2606.02784>
- [32] Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich Küttler, Mike Lewis, Wen-tau Yih, Tim Rocktäschel, Sebastian Riedel, and Douwe Kiela. 2020. Retrieval-augmented generation for knowledge-intensive NLP tasks. In *Proceedings of the 34th International Conference on Neural Information Processing Systems (Vancouver, BC, Canada) (NIPS '20)*. Curran Associates Inc., Red Hook, NY, USA, Article 793, 16 pages.
- [33] Hui Li, Shiyuan Deng, Xiao Yan, Xiangyu Zhi, and James Cheng. 2025. SAQ: Pushing the Limits of Vector Quantization through Code Adjustment and Dimension Segmentation. *Proc. ACM Manag. Data* 3, 6, Article 359 (Dec. 2025), 25 pages. <https://doi.org/10.1145/3769824>
- [34] Huiling Li, Xin Huang, Byron Choi, and Jianliang Xu. 2026. BAMG: A Block-Aware Monotonic Graph Index for Disk-Based Approximate Nearest Neighbor Search. arXiv:2509.03226 [cs.DB] <https://arxiv.org/abs/2509.03226>
- [35] Liang Li, Shufeng Gong, Yanan Yang, Yiduo Wang, and Jie Wu. 2026. I/O Optimizations for Graph-Based Disk-Resident Approximate Nearest Neighbor Search: A Design Space Exploration. *arXiv preprint arXiv:2602.21514* (2026).
- [36] Edo Liberty, Ashwin Padaki, and Amir Ingber. [n.d.]. VQ-bench: A Composable Vector Quantization Framework. In *The 2nd Workshop on Vector Databases*.
- [37] Linux man-pages project. [n.d.]. open(2) — Linux Manual Page. <https://man7.org/linux/man-pages/man2/open.2.html>. Accessed 2026-07.
- [38] Julieta Martinez, Shobhit Zakhmi, Holger H. Hoos, and James J. Little. 2018. LSQ++: Lower Running Time and Higher Recall in Multi-Codebook Quantization. In *Computer Vision – ECCV 2018: 15th European Conference, Munich, Germany, September 8–14, 2018, Proceedings, Part XVI* (Munich, Germany). Springer-Verlag, Berlin, Heidelberg, 508–523. https://doi.org/10.1007/978-3-030-01270-0_30
- [39] Yusuke Matsui, Yusuke Uchida, Hervé Jégou, and Shin'ichi Satoh. 2018. A Survey of Product Quantization. *ITE Transactions on Media Technology and Applications* 6, 1 (2018), 2–10.
- [40] Meta AI. 2025. The DINO-10B Dataset. <https://github.com/facebookresearch/faiss/blob/main/contrib/datasets.py>
- [41] Microsoft. [n.d.]. DiskANN: Graph-structured Indices for Scalable, Fast, Fresh and Filtered Approximate Nearest Neighbor Search. https://github.com/microsoft/DiskANN.cpp_main_branch
- [42] Jiongkang Ni, Xiaoliang Xu, Yuxiang Wang, Can Li, Jiajie Yao, Shihai Xiao, and Xuechang Zhang. 2023. DiskANN++: Efficient Page-based Search over Isomorphic Mapped Graph Index using Query-sensitivity Entry Vertex. arXiv:2310.00402 [cs.IR] <https://arxiv.org/abs/2310.00402>
- [43] OpenAI. 2024. New embedding models and API updates. <https://openai.com/index/new-embedding-models-and-api-updates/>. Accessed: 2026-06-08.
- [44] Yu Pan, Jianxin Sun, and Hongfeng Yu. 2023. LM-DiskANN: Low Memory Footprint in Disk-Native Dynamic Graph-Based ANN Indexing. In *2023 IEEE International Conference on Big Data (BigData)*. 5987–5996. <https://doi.org/10.1109/BigData59044.2023.10386517>
- [45] John Paparrizos, Ikradya Edian, Chunwei Liu, Aaron J. Elmore, and Michael J. Franklin. 2022. Fast Adaptive Similarity Search through Variance-Aware Quantization. In *2022 IEEE 38th International Conference on Data Engineering (ICDE)*. 2969–2983. <https://doi.org/10.1109/ICDE53745.2022.00268>
- [46] RAPIDS Development Team. 2024. The Wiki-all Dataset. https://docs.rapids.ai/api/cuvs/stable/cuvs_bench/wiki_all_dataset/. 88M multilingual Wikipedia passage embeddings with 1M/10M subsets; part of the cuVS benchmark suite. Accessed July 2026.

- [47] Hanan Samet. 2005. *Foundations of Multidimensional and Metric Data Structures (The Morgan Kaufmann Series in Computer Graphics and Geometric Modeling)*. Morgan Kaufmann Publishers Inc., San Francisco, CA, USA.
- [48] Jifan Shi, Jianyang Gao, James Xia, Tamás Béla Fehér, and Cheng Long. 2026. GPU-Native Approximate Nearest Neighbor Search with IVF-RaBitQ: Fast Index Build and Search. arXiv:2602.23999 [cs.DB] <https://arxiv.org/abs/2602.23999>
- [49] Aditi Singh, Suhas Jayaram Subramanya, Ravishankar Krishnaswamy, and Harsha Vardhan Simhadri. 2021. FreshDiskANN: A Fast and Accurate Graph-Based ANN Index for Streaming Similarity Search. CoRR abs/2105.09613 (2021). arXiv:2105.09613 <https://arxiv.org/abs/2105.09613>
- [50] SISAP Challenge Committee. 2024. The LAION2B and projections: SISAP 2024 Indexing Challenge datasets. <https://sisap-challenges.github.io/2024/datasets/>. Accessed: 2026-06-08.
- [51] Jaeyong Song, Hongsun Jang, Changmin Shin, Seongyeon Park, Yong Jae Ryou, Seo Jin Park, and Jinho Lee. 2026. NAVIS: Concurrent Search and Update with Low Position-Seeking Overhead in On-SSD Graph-Based Vector Search. arXiv:2605.11523 [cs.DB] <https://arxiv.org/abs/2605.11523>
- [52] Ziwen Song, Bin Wang, Xiaochun Yang, and Junhua Zhang. 2026. Low-Latency Out-of-Core ANN Search in High-Dimensional Space. arXiv:2605.05787 [cs.DB] <https://arxiv.org/abs/2605.05787>
- [53] Suhas Jayaram Subramanya, Devvrit, Rohan Kadekodi, Ravishankar Krishnaswamy, and Harsha Vardhan Simhadri. 2019. *DiskANN: fast accurate billion-point nearest neighbor search on a single node*. Curran Associates Inc., Red Hook, NY, USA.
- [54] Kento Tatsuno, Daisuke Miyashita, Taiga Ikeda, Kiyoshi Ishiyama, Kazunari Sumiyoshi, and Jun Deguchi. 2025. AiSAQ: All-in-Storage ANNS with Product Quantization for DRAM-free Information Retrieval. arXiv:2404.06004 [cs.IR] <https://arxiv.org/abs/2404.06004>
- [55] THU Storage Group. [n.d.]. PipeANN: A Low-latency, Billion-scale, and Updatable Graph-based Vector Store on SSD. <https://github.com/thustorage/PipeANN>.
- [56] Ertem Tuncel, Hakan Ferhatosmanoglu, and Kenneth Rose. 2002. VQ-Index: An Index Structure for Similarity Searching in Multimedia Databases. In *Proceedings of the Tenth ACM International Conference on Multimedia (Juan-les-Pins, France) (MULTIMEDIA '02)*. Association for Computing Machinery, New York, NY, USA, 543–552. <https://doi.org/10.1145/641007.641117>
- [57] Shay Vargaftik, Ran Ben Basat, Amit Portnoy, Gal Mendelson, Yaniv Ben-Itzhak, and Michael Mitzenmacher. 2022. EDEN: Communication-Efficient and Robust Distributed Mean Estimation for Federated Learning. arXiv:2108.08842 [cs.LG] <https://arxiv.org/abs/2108.08842>
- [58] Shay Vargaftik, Ran Ben-Basat, Amit Portnoy, Gal Mendelson, Yaniv Ben-Itzhak, and Michael Mitzenmacher. 2021. Drive: One-bit distributed mean estimation. *Advances in Neural Information Processing Systems* 34 (2021), 362–377.
- [59] VectorDB-NTU. 2025. The RaBitQ Library. <https://github.com/VectorDB-NTU/RaBitQ-Library>.
- [60] Jun Wang, Wei Liu, Sanjiv Kumar, and Shih-Fu Chang. 2016. Learning to Hash for Indexing Big Data - A Survey. *Proc. IEEE* 104, 1 (2016), 34–57. <https://doi.org/10.1109/JPROC.2015.2487976>
- [61] Jianguo Wang, Xiaomeng Yi, Rentong Guo, Hai Jin, Peng Xu, Shengjun Li, Xiangyu Wang, Xiangzhou Guo, Chengming Li, Xiaohai Xu, Kun Yu, Yuxing Yuan, Yinghao Zou, Jiquan Long, Yudong Cai, Zhenxiang Li, Zhifeng Zhang, Yihua Mo, Jun Gu, Ruiyi Jiang, Yi Wei, and Charles Xie. 2021. Milvus: A Purpose-Built Vector Data Management System. In *Proceedings of the 2021 International Conference on Management of Data (Virtual Event, China) (SIGMOD '21)*. Association for Computing Machinery, New York, NY, USA, 2614–2627. <https://doi.org/10.1145/3448016.3457550>
- [62] Jingdong Wang, Ting Zhang, jingkuang song, Nicu Sebe, and Heng Tao Shen. 2018. A Survey on Learning to Hash. *IEEE Transactions on Pattern Analysis and Machine Intelligence* 40, 4 (2018), 769–790. <https://doi.org/10.1109/TPAMI.2017.2699960>
- [63] Mengzhao Wang, Weizhi Xu, Xiaomeng Yi, Songlin Wu, Zhangyang Peng, Xiangyu Ke, Yunjun Gao, Xiaoliang Xu, Rentong Guo, and Charles Xie. 2024. Starling: An I/O-Efficient Disk-Resident Graph Index Framework for High-Dimensional Vector Similarity Search on Data Segment. *Proc. ACM Manag. Data* 2, 1, Article 14 (March 2024), 27 pages. <https://doi.org/10.1145/3639269>
- [64] Roger Weber, Hans-Jörg Schek, and Stephen Blott. 1998. A Quantitative Analysis and Performance Study for Similarity-Search Methods in High-Dimensional Spaces. In *Proceedings of the 24rd International Conference on Very Large Data Bases (VLDB '98)*. Morgan Kaufmann Publishers Inc., San Francisco, CA, USA, 194–205.
- [65] Chuangxian Wei, Bin Wu, Sheng Wang, Renjie Lou, Chaoqun Zhan, Feifei Li, and Yuanzhe Cai. 2020. AnalyticDB-V: a hybrid analytical engine towards query fusion for structured and unstructured data. *Proc. VLDB Endow.* 13, 12 (Aug. 2020), 3152–3165. <https://doi.org/10.14778/3415478.3415541>
- [66] Yuming Xu, Hengyu Liang, Jin Li, Shuotao Xu, Qi Chen, Qianxi Zhang, Cheng Li, Ziyue Yang, Fan Yang, Yuqing Yang, Peng Cheng, and Mao Yang. 2023. SPFresh: Incremental In-Place Update for Billion-Scale Vector Search. In *Proceedings of the 29th Symposium on Operating Systems Principles (Koblenz, Germany) (SOSP '23)*. Association for Computing Machinery, New York, NY, USA, 545–561. <https://doi.org/10.1145/3600006.3613166>
- [67] Mingyu Yang, Liuchang Jing, Wentao Li, and Wei Wang. 2026. Quantization Meets Projection: A Happy Marriage for Approximate k-Nearest Neighbor Search. arXiv:2411.06158 [cs.DB] <https://arxiv.org/abs/2411.06158>
- [68] Peiqi Yin, Xiao Yan, Qihui Zhou, Hui Li, Xiaolu Li, Lin Zhang, Meiling Wang, Xin Yao, and James Cheng. 2025. Gorgeous: Revisiting the Data Layout for Disk-Resident High-Dimensional Vector Search. arXiv:2508.15290 [cs.DB] <https://arxiv.org/abs/2508.15290>
- [69] Ziyang Yue, Bolong Zheng, Ling Xu, Kanru Xu, Shuhao Zhang, Yajuan Du, Yunjun Gao, Xiaofang Zhou, and Christian S. Jensen. 2025. Select Edges Wisely: Monotonic Path Aware Graph Layout Optimization for Disk-Based ANN Search. *Proc. VLDB Endow.* 18, 11 (July 2025), 4337–4349. <https://doi.org/10.14778/3749646.3749697>
- [70] Amir Zandieh, Majid Daliri, Majid Hadian, and Vahab Mirrokni. 2025. TurboQuant: Online Vector Quantization with Near-optimal Distortion Rate. arXiv:2504.19874 [cs.LG] <https://arxiv.org/abs/2504.19874>
- [71] Pavel Zezula, Giuseppe Amato, Vlastislav Dohnal, and Michal Batko. 2010. *Similarity Search: The Metric Space Approach* (1st ed.). Springer Publishing Company, Incorporated.
- [72] Ting Zhang, Chao Du, and Jingdong Wang. 2014. Composite Quantization for Approximate Nearest Neighbor Search. In *Proceedings of the 31st International Conference on Machine Learning (Proceedings of Machine Learning Research)*, Eric P. Xing and Tony Jebara (Eds.), Vol. 32. PMLR, Beijing, China, 838–846. <https://proceedings.mlr.press/v32/zhangd14.html>
- [73] Weichen Zhao, Yuncheng Lu, Hao Tian, Hao Zhang, Jiehui Li, Minghao Zhao, Yakun Li, and Weining Qian. 2026. Optimizing SSD-Resident Graph Indexing for High-Throughput Vector Search. arXiv:2602.22805 [cs.DB] <https://arxiv.org/abs/2602.22805>
- [74] Shurui Zhong, Dingheng Mo, and Siqiang Luo. 2026. LSM-VEC: A Large-Scale Disk-Based System for Dynamic Vector Search. arXiv:2505.17152 [cs.DB] <https://arxiv.org/abs/2505.17152>
- [75] Yijie Zhou, Shengyuan Lin, Shufeng Gong, Song Yu, Shuhao Fan, Yanfeng Zhang, and Ge Yu. 2025. GoVector: An I/O-Efficient Caching Strategy for High-Dimensional Vector Nearest Neighbor Search. arXiv:2508.15694 [cs.DB] <https://arxiv.org/abs/2508.15694>
- [76] Zilliz. 2024. Starling. <https://github.com/zilliztech/starling>.

A Analysis of Split-RaBitQ

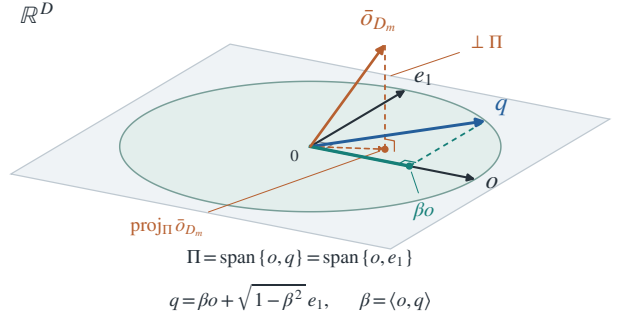


Figure 11: Geometry of Split-RaBitQ in $\mathbb{R}^D$. The shaded plane $\Pi = \text{span}\{o, q\}$ contains the orthogonal unit vectors o and e_1 , and the unit query q ; the ellipse depicts its unit circle. The prefix vector $\bar{o}_{D_m}$, of norm $\sqrt{D_m/D}$, is projected onto Π . Its component perpendicular to Π contributes nothing to $\langle \bar{o}_{D_m}, q \rangle$.

Throughout this appendix, all probabilities and expectations are over the random orthogonal matrix P , which, as in [18], is generated by Gram–Schmidt orthonormalization of a matrix with independent standard Gaussian entries; $P^{-1}o$ then follows the uniform distribution on the unit sphere (Section B.2 of [18]). The constants c_0 and c are those in the coordinate tail bound (32) and the Lipschitz concentration inequality (41) of [18], and, by the expression in Lemma B.3 of [18], $\mathbb{E}\langle \bar{o}, o \rangle$ exceeds 0.79 for every $D \geq 2$. Eq. (1),

i.e., (30) of [18], applied to $\bar{x}$ and to $\bar{x}_{D_m}$ gives

$$\langle \bar{o}, o \rangle = \frac{1}{\sqrt{D}} \sum_{i=1}^D |(P^{-1}o)[i]|, \quad \langle \bar{o}_{D_m}, o \rangle = \frac{1}{\sqrt{D}} \sum_{i=1}^{D_m} |(P^{-1}o)[i]|, \quad (4)$$

since $\bar{x}[i] = \text{sign}((P^{-1}o)[i])/\sqrt{D}$ and $\bar{x}_{D_m}$ keeps the first D_m of these entries. In particular $\langle \bar{o}, o \rangle \geq 1/\sqrt{D}$, because the ℓ_1 norm of the unit vector $P^{-1}o$ is at least its ℓ_2 norm.

A.1 Joint Distribution of the Prefix's Components

Lemma B.3 of [18] states that the joint distribution of $(\langle \bar{o}, o \rangle, \langle \bar{o}, e_1 \rangle)$ is identical to that of $(\langle \bar{o}, o \rangle, \sqrt{1 - \langle \bar{o}, o \rangle^2} X_1)$, where X_1 follows the distribution p_{D-1} of Lemma B.1 in [18] and is independent of $\langle \bar{o}, o \rangle$. The following lemma extends it to the prefix.

LEMMA A.1. *Let o and e_1 be two unit vectors with $o \perp e_1$, and let $\bar{x}$, $\bar{o}$, $\bar{x}_{D_m}$, and $\bar{o}_{D_m}$ be as in §3.1. The joint distribution of $(\langle \bar{o}, o \rangle, \langle \bar{o}_{D_m}, o \rangle, \langle \bar{o}_{D_m}, e_1 \rangle)$ is identical to that of*

$$(\langle \bar{o}, o \rangle, \langle \bar{o}_{D_m}, o \rangle, \sqrt{D_m/D - \langle \bar{o}_{D_m}, o \rangle^2} X_1), \quad (5)$$

where X_1 follows the distribution p_{D-1} of Lemma B.1 in [18] and is independent of $P^{-1}o$, hence of the first two components.

PROOF. We follow the proof of Lemma B.3 in [18] step by step, with $\bar{x}_{D_m}$ in place of $\bar{x}$ where the two differ.

Reduction. Let $u_1 := (1, 0, \dots, 0)$ and $u_2 := (0, 1, 0, \dots, 0)$. As in [18], since the distribution of UP is identical to that of P for every orthogonal matrix U , it suffices to prove the lemma for $o = u_1$ and $e_1 = u_2$. Let p_1 and p_2 be the first two rows of P , as in [18]; then $P^{-1}o = P^T u_1 = p_1$ and $P^{-1}e_1 = p_2$.

Deferred decision. The rows of P are generated one by one ((46) of [18]): p_1 is a normalized standard Gaussian vector, and $p_2 = (g - \langle g, p_1 \rangle p_1) / \|g - \langle g, p_1 \rangle p_1\|$ for a standard Gaussian vector g independent of p_1 . Once p_1 is sampled, $\bar{x} = \arg \max_{x \in C} \langle x, p_1 \rangle$ is fixed ((47)–(48) of [18]), and so is $\bar{x}_{D_m}$, obtained from $\bar{x}$ by setting its coordinates beyond the first D_m to zero, an operation that does not involve P . Hence $\langle \bar{o}, o \rangle = \langle \bar{x}, p_1 \rangle$ and $\langle \bar{o}_{D_m}, o \rangle = \langle \bar{x}_{D_m}, p_1 \rangle$ are functions of p_1 , whereas $\langle \bar{o}_{D_m}, e_1 \rangle = \langle P \bar{x}_{D_m}, u_2 \rangle = \langle \bar{x}_{D_m}, p_2 \rangle$ involves p_2 as well. It remains to determine the conditional distribution of $\langle \bar{x}_{D_m}, p_2 \rangle$ given p_1 .

Alignment. Conditional on p_1 , the vector $\bar{x}_{D_m}$ is fixed, with $\|\bar{x}_{D_m}\|^2 = D_m/D$ (it has D_m entries of magnitude $1/\sqrt{D}$). Let

$$v_1 := (1, 0, \dots, 0), \\ v_2 := (\langle \bar{x}_{D_m}, p_1 \rangle, \sqrt{D_m/D - \langle \bar{x}_{D_m}, p_1 \rangle^2}, 0, \dots, 0).$$

The pairs $(p_1, \bar{x}_{D_m})$ and (v_1, v_2) have the same norms and the same inner product, so there is an orthogonal matrix V with $Vp_1 = v_1$ and $V\bar{x}_{D_m} = v_2$. This is the alignment step of [18], where $\bar{x}$ has unit norm and the second entry of v_2 is $\sqrt{1 - \langle \bar{x}, p_1 \rangle^2}$; for the prefix the

norm is $\sqrt{D_m/D}$. Then, as in (49)–(51) of [18],

$$\langle \bar{x}_{D_m}, p_2 \rangle = \left\langle \bar{x}_{D_m}, \frac{g - \langle g, p_1 \rangle p_1}{\|g - \langle g, p_1 \rangle p_1\|} \right\rangle \quad (6)$$

$$= \left\langle V \bar{x}_{D_m}, \frac{Vg - \langle Vg, Vp_1 \rangle Vp_1}{\|Vg - \langle Vg, Vp_1 \rangle Vp_1\|} \right\rangle \quad (7)$$

$$= \left\langle v_2, \frac{g - \langle g, v_1 \rangle v_1}{\|g - \langle g, v_1 \rangle v_1\|} \right\rangle, \quad (8)$$

where (6) is the Gram–Schmidt step, (7) holds because inner products and norms are invariant under V , and (8) substitutes $Vp_1 = v_1$, $V\bar{x}_{D_m} = v_2$, and Vg by g , since the standard Gaussian vector is rotation invariant and g is independent of p_1 . The vector $g - \langle g, v_1 \rangle v_1$ has first entry 0 and its remaining $D - 1$ entries are independent standard Gaussians; after normalization, the remaining $D - 1$ entries follow the uniform distribution on the unit sphere in the $(D - 1)$ -dimensional space and are independent of p_1 . Recall that $v_2 = (\langle \bar{x}_{D_m}, p_1 \rangle, \sqrt{D_m/D - \langle \bar{x}_{D_m}, p_1 \rangle^2}, 0, \dots, 0)$. Thus, $\langle \bar{x}_{D_m}, p_2 \rangle = \sqrt{D_m/D - \langle \bar{x}_{D_m}, p_1 \rangle^2} \cdot X_1$, where X_1 follows the distribution p_{D-1} and is independent of p_1 . Since $\langle \bar{x}_{D_m}, p_2 \rangle = \langle \bar{o}_{D_m}, e_1 \rangle$ and $\langle \bar{x}_{D_m}, p_1 \rangle = \langle \bar{o}_{D_m}, o \rangle$, this is the third component of Eq. (5), and the independence of X_1 from p_1 gives its independence from the first two components, which are functions of p_1 . $\square$

A.2 Concentration of the Prefix's Share

Lemma B.2 of [18] states that for an L -Lipschitz function f on the unit sphere and a uniformly random unit vector x , $\Pr\{|f(x) - \mathbb{E}f(x)| \geq t\} \leq 2 \exp(-cDt^2/L^2)$ ((41) there), and applies it with $f(x) = \|x\|_1/\sqrt{D}$ and $x = P^{-1}o$ to obtain, for every $u > 0$,

$$\Pr\{|\langle \bar{o}, o \rangle - \mathbb{E}\langle \bar{o}, o \rangle| \geq u/\sqrt{D}\} \leq 2e^{-cu^2} \quad (9)$$

((54) there). The same inequality controls the prefix's share.

LEMMA A.2. *For every $u > 0$,*

$$\Pr\left\{\left|\langle \bar{o}_{D_m}, o \rangle - \frac{D_m}{D} \mathbb{E}\langle \bar{o}, o \rangle\right| \geq \frac{u\sqrt{D_m}}{D}\right\} \leq 2e^{-cu^2}.$$

PROOF. By Eq. (4), $\langle \bar{o}_{D_m}, o \rangle = f(P^{-1}o)$ with $f(x) := \frac{1}{\sqrt{D}} \sum_{i=1}^{D_m} |x[i]|$, and $P^{-1}o$ is a uniformly random unit vector, so its coordinates are exchangeable and $\mathbb{E}|(P^{-1}o)[i]|$ does not depend on i . By Eq. (4),

$$\begin{aligned} \mathbb{E}f(P^{-1}o) &= \frac{D_m}{\sqrt{D}} \mathbb{E}|(P^{-1}o)[1]| \\ &= \frac{D_m}{D} \cdot \frac{1}{\sqrt{D}} \sum_{i=1}^D \mathbb{E}|(P^{-1}o)[i]| \\ &= \frac{D_m}{D} \mathbb{E}\langle \bar{o}, o \rangle. \end{aligned}$$

For unit vectors x and y , $|f(x) - f(y)| \leq \frac{1}{\sqrt{D}} \sum_{i=1}^{D_m} |x[i] - y[i]| \leq \sqrt{D_m/D} \|x - y\|$ by the triangle inequality and the Cauchy–Schwarz inequality, so f is $\sqrt{D_m/D}$ -Lipschitz on the unit sphere. Lemma B.2 of [18] with $L = \sqrt{D_m/D}$ and $t = u\sqrt{D_m}/D$ gives the bound, since $cDt^2/L^2 = cu^2$. $\square$

A.3 Proof of Theorem 3.1

Recall Eq. (3): with $\beta = \langle o, q \rangle$ and $A_{D_m} = (D/D_m) \langle \bar{o}_{D_m}, o \rangle / \langle \bar{o}, o \rangle$, the error of the estimator is $\beta(A_{D_m} - 1) + \sqrt{1 - \beta^2} (D/D_m) \langle \bar{o}_{D_m}, e_1 \rangle / \langle \bar{o}, o \rangle$. When o and q are collinear, only the first term is present.

Unbiasedness. We show that both terms have mean zero. For the first term, by Eq. (4),

$$A_{D_m} = \frac{D}{D_m} \cdot \frac{\sum_{i=1}^{D_m} |(P^{-1}o)[i]|}{\sum_{j=1}^D |(P^{-1}o)[j]|}. \quad (10)$$

The coordinates of the uniformly random unit vector $P^{-1}o$ are exchangeable, hence so are the ratios $| (P^{-1}o)[i] | / \sum_{j=1}^D |(P^{-1}o)[j]|$, $i = 1, \dots, D$, which sum to 1; each ratio therefore has expectation $1/D$, the sum of the first D_m of them has expectation D_m/D , and $\mathbb{E}[A_{D_m}] = 1$. (This uses that the prefix is a fixed set of coordinates, chosen without reference to P .) For the second term, Lemma A.1 gives

$$\mathbb{E} \left[\frac{D}{D_m} \frac{\langle \bar{o}_{D_m}, e_1 \rangle}{\langle \bar{o}, o \rangle} \right] = \mathbb{E} \left[\frac{D}{D_m} \frac{\sqrt{D_m/D - \langle \bar{o}_{D_m}, o \rangle^2}}{\langle \bar{o}, o \rangle} X_1 \right] \quad (11)$$

$$= \mathbb{E} \left[\frac{D}{D_m} \frac{\sqrt{D_m/D - \langle \bar{o}_{D_m}, o \rangle^2}}{\langle \bar{o}, o \rangle} \right] \cdot \mathbb{E}[X_1] \quad (12)$$

$$= 0, \quad (13)$$

where (11) is by Lemma A.1, (12) is due to the independence between X_1 and $P^{-1}o$ (the first factor is a function of $P^{-1}o$, bounded because $\langle \bar{o}, o \rangle \geq 1/\sqrt{D}$), and (13) is because p_{D-1} has mean 0. Multiplied by $\sqrt{1 - \beta^2}$, the second term has mean zero, which completes the proof of the unbiasedness.

Error bound. By Lemma A.1, the error has the same distribution as

$$Z := \beta(A_{D_m} - 1) + \sqrt{1 - \beta^2} \frac{D}{D_m} \frac{\sqrt{D_m/D - \langle \bar{o}_{D_m}, o \rangle^2}}{\langle \bar{o}, o \rangle} X_1, \quad (14)$$

so it suffices to bound Z . Fix $0 < \epsilon \leq \sqrt{D_m}/3$ and consider three events:

- $\mathcal{E}_1$: $|\langle \bar{o}_{D_m}, o \rangle - (D_m/D) \mathbb{E}\langle \bar{o}, o \rangle| \leq \epsilon \sqrt{D_m}/D$. By Lemma A.2 with $u = \epsilon$, $\Pr\{\mathcal{E}_1^c\} \leq 2e^{-c\epsilon^2}$.
- $\mathcal{E}_2$: $|\langle \bar{o}, o \rangle - \mathbb{E}\langle \bar{o}, o \rangle| \leq \epsilon/\sqrt{D}$. By Eq. (9) with $u = \epsilon$, $\Pr\{\mathcal{E}_2^c\} \leq 2e^{-c\epsilon^2}$. On $\mathcal{E}_2$, $\langle \bar{o}, o \rangle \geq \mathbb{E}\langle \bar{o}, o \rangle/2$, because $\epsilon/\sqrt{D} \leq \epsilon/\sqrt{D_m} \leq 1/3 < 0.79/2 \leq \mathbb{E}\langle \bar{o}, o \rangle/2$.
- $\mathcal{E}_3$: $|X_1| \leq \epsilon/\sqrt{D-1}$. By the tail bound (32) of [18] in dimension $D-1$, $\Pr\{\mathcal{E}_3^c\} \leq 2e^{-c_0\epsilon^2}$.

On $\mathcal{E}_1 \cap \mathcal{E}_2$, by Eq. (4) and the triangle inequality,

$$\begin{aligned} |A_{D_m} - 1| &= \frac{|(D/D_m) \langle \bar{o}_{D_m}, o \rangle - \langle \bar{o}, o \rangle|}{\langle \bar{o}, o \rangle} \\ &\leq \frac{(D/D_m) \cdot \epsilon \sqrt{D_m}/D + \epsilon/\sqrt{D}}{\mathbb{E}\langle \bar{o}, o \rangle/2} \leq \frac{4}{\mathbb{E}\langle \bar{o}, o \rangle} \cdot \frac{\epsilon}{\sqrt{D_m}}, \end{aligned}$$

where the numerator bounds $|(D/D_m) \langle \bar{o}_{D_m}, o \rangle - \mathbb{E}\langle \bar{o}, o \rangle|$ by $\mathcal{E}_1$ and $|\mathbb{E}\langle \bar{o}, o \rangle - \langle \bar{o}, o \rangle|$ by $\mathcal{E}_2$, and $\epsilon/\sqrt{D} \leq \epsilon/\sqrt{D_m}$ gives the last step.

On $\mathcal{E}_2 \cap \mathcal{E}_3$, the second term of Z is at most (since $\sqrt{D_m/D - \langle \bar{o}_{D_m}, o \rangle^2} \leq \sqrt{D_m/D}$ and $\sqrt{1 - \beta^2} \leq 1$)

$$\begin{aligned} \frac{D}{D_m} \sqrt{\frac{D_m}{D}} \cdot \frac{\epsilon/\sqrt{D-1}}{\mathbb{E}\langle \bar{o}, o \rangle/2} &= \frac{2}{\mathbb{E}\langle \bar{o}, o \rangle} \sqrt{\frac{D}{D-1}} \cdot \frac{\epsilon}{\sqrt{D_m}} \\ &\leq \frac{2\sqrt{2}}{\mathbb{E}\langle \bar{o}, o \rangle} \cdot \frac{\epsilon}{\sqrt{D_m}} \end{aligned}$$

in magnitude, using $D/(D-1) \leq 2$. Hence, with $|\beta| \leq 1$, on $\mathcal{E}_1 \cap \mathcal{E}_2 \cap \mathcal{E}_3$,

$$|Z| \leq \frac{4 + 2\sqrt{2}}{\mathbb{E}\langle \bar{o}, o \rangle} \cdot \frac{\epsilon}{\sqrt{D_m}} < \frac{9\epsilon}{\sqrt{D_m}},$$

the last step because $\mathbb{E}\langle \bar{o}, o \rangle > 0.79$ and $(4 + 2\sqrt{2})/0.79 < 9$. By the union bound,

$$\Pr\{|Z| > 9\epsilon/\sqrt{D_m}\} \leq \sum_{j=1}^3 \Pr\{\mathcal{E}_j^c\} \leq 4e^{-c\epsilon^2} + 2e^{-c_0\epsilon^2} \leq 6e^{-c'\epsilon^2}, \quad (15)$$

where $c' := \min(c_0, c)$. Since a probability is at most 1, the right-hand side of Eq. (15) can be replaced by $2e^{-c''\epsilon^2}$ with $c'' := c' \ln 2 / \ln 6$: when $c'\epsilon^2 \leq \ln 6$ the new bound is at least 1, and otherwise $6e^{-c'\epsilon^2} \leq 2e^{-c''\epsilon^2}$ because $(c' - c'')\epsilon^2 = (c' \ln 3 / \ln 6) \epsilon^2 > \ln 3$. Finally, applying the result with $\epsilon/9$ in place of ϵ (which stays within the range $\epsilon/9 \leq \sqrt{D_m}/3$) gives $\Pr\{|Z| > \epsilon/\sqrt{D_m}\} \leq 2e^{-c_1\epsilon^2}$ with $c_1 := c''/81$, which is the error bound of Theorem 3.1.

A.4 Empirical Error Distribution

We examine the estimation error for synthetic unit-vector pairs in $\mathbb{R}^{768}$ with prescribed inner products $\beta = \langle o, q \rangle \in \{0.5, 0.9\}$. For each fixed pair, we evaluate both $D_m = D$ and $D_m = D/2$ over 10^6 independent Haar-distributed rotations, which preserve β while varying the quantization error. Figure 12 shows error distributions centered near zero, consistent with the estimator's unbiasedness. Retaining half the coordinates broadens both distributions, reflecting the additional uncertainty introduced by the prefix.

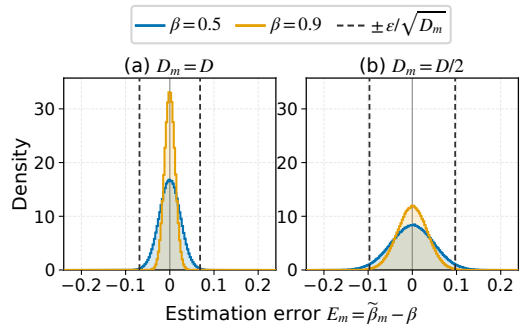


Figure 12: Empirical estimation-error distributions for $D = 768$ with (a) $D_m = D$ and (b) $D_m = D/2$. Dashed lines mark $\pm\epsilon/\sqrt{D_m}$ for $\epsilon = 1.9$.

We use $\epsilon = 1.9$, following RaBitQ [17], and count a lower-bound violation when $\hat{\beta}_{D_m} - \beta < -\epsilon/\sqrt{D_m}$. For $\beta = 0.5$, the violation rate increases from 0.18% at $D_m = D$ to 2.17% at $D_m = D/2$. The errors at $\beta = 0.9$ are more concentrated: no violations are observed with

Table 7: Search-result determinism on YFCC-100M ($T = 64$). Percentages give the fraction of queries with unchanged results across all eight runs, treating substitutions between points at the same true distance to the query as unchanged. The rank ranges refer to positions in the returned list, with $r = 0.4k$ for both variants; order within each set is ignored. Recall spread is the difference between the maximum and minimum recall across runs, in percentage points (pp).

k	Variant	Queries with unchanged results (%)			Recall spread (pp)
		Top- k	$[1, r]$	$(r, k]$	
10	Scheduled	99.95	100.00	99.95	0.003
	Full	97.89	99.98	97.89	0.022
100	Scheduled	99.96	100.00	99.96	0.0002
	Full	35.85	99.66	35.85	0.645

the full code in 10^6 trials, while the half-length prefix yields a rate of 0.21%.

B Search-Result Determinism

We examine search-result determinism using the n_{probe} selected for Full to achieve approximately 0.90 recall, keeping the same value for Scheduled. We execute the query workload eight times, three in the original order and five with shuffled query orders. Table 7 compares the returned neighbor sets across these runs.

Scheduled is nearly deterministic after accounting for distance ties. Among 10,000 queries, only 5 at $k = 10$ and 4 at $k = 100$ return different top- k results across the eight runs. The argument of search-result determinism assumes exact re-ranking and valid lower bounds. The remaining changes occur near the rank- k boundary, where the affected candidates have lower bounds above their true distances and the final k -th distances in runs that omit them. Such violations reflect the probabilistic nature of the lower bound, also illustrated by the empirical error distributions in §A.4. Although a candidate’s lower bound is fixed across runs, the candidate may be re-ranked before the running threshold falls below that bound, or pruned if tested later. Recall varies by at most 0.003 percentage points across runs.

Full’s output varies mainly in the tail. At $k = 100$, the unchanged-query fraction is 35.85% for both the entire top- k and ranks $(0.4k, k]$, compared with 99.66% for the first 40 positions. This pattern is consistent with the α -quantile threshold making retention of neighbors at ranks $(\alpha k, k]$ sensitive to I/O completion order. Earlier re-ranking of closer candidates can tighten the threshold before these neighbors’ pages are issued, whereas a different completion order may allow those pages to be read. Although I/O completion order affects the returned neighbors, recall remains relatively stable across the eight runs, ranging from 0.9005 to 0.9069 at $k = 100$.

C Additional Evaluation Results

C.1 Main Evaluation

Table 8 reports throughput at recall 0.90 and 0.95 for all five datasets in the main evaluation (§5.2).

C.2 Ablation Study

Figure 13 shows the throughput–recall curves for the three ablations on YFCC-100M and LAION-100M.

On YFCC-100M, RaBitQ-SSD ($B = 9$) is faster than DiskANN (disk-PQ) at recall 0.90, but the ordering reverses at recall 0.95 (Figure 13(b)). Reaching the higher recall requires more probing, increasing the work spent scanning full-dimensional in-memory codes. FastScan becomes the main bottleneck for RaBitQ-SSD, limiting the throughput benefit of fewer SSD reads.

The gap between $B = 5$ and SQ4 grows at higher recall on both datasets (Figure 13(c,f)). SQ4 requires substantially more probing to reach higher recall, reducing throughput. In the measured sweep, SQ4 reaches recall 0.95 on YFCC-100M at much lower throughput than $B = 5$, and does not reach recall 0.95 on LAION-100M.

C.3 Parameter Study

The in-flight budget Q . Figure 14 reports throughput at recall 0.90 against Q on YFCC-100M, one line per thread count ($D_m = D$). When Q is too small, a thread has too few reads in flight to use the SSD’s internal parallelism. At $k = 1000$, where a query issues hundreds of reads, throughput at $T = 1$ falls to about a third as Q goes from 16 to 4; at $k = 10$ the same change barely changes throughput, because the few reads hide behind compute, as the $T = 1$ timeline of Fig. 4 showed. When Q is too large, the pops reach deeper into the priority set and commit candidates that a slightly later threshold would have pruned (§4.3). Each such commit reads a page the query did not need: the page is fetched, swept, and re-screened, some of its members are re-ranked, and none of that work would have happened at a smaller Q . Under a heavy I/O load the excess reads also lengthen completion latency (§4.4), so candidates are committed against a staler threshold and fewer of them are pruned. Past $Q = 16$ throughput declines slightly at $T = 1$; at $T = 64$ and $k = 10$ it falls to about a third of its value at the best Q . The best Q therefore rises with k and falls with T , and the deployed rule ($Q = 16$ for $T \leq 16$, $Q = 8$ above) stays within 3.7% of the throughput at the best Q at every T and k measured.

The drain threshold α . Figure 15 reports throughput against recall for $\alpha \in \{0.1, 0.2, 0.3, 0.4, 0.5, 0.6, 0.8, 1.0\}$ on YFCC-100M at $T = 64$ and $D_m = D$. $\alpha = 1$ recovers Scheduled (§4.4); $\alpha = 0.4$ is the default. A tighter α prunes more of the pages left pending when the scan ends, so a query reads fewer of them: at recall 0.90 and $k = 100$, the pages per query are 125 at $\alpha = 0.1$, 132 at the default, and 236 at $\alpha = 1$; at $k = 1000$ they are 893, 957, and 1,699.

Below the default the count barely moves, while above it nearly doubles. At recall 0.90 the default stays within 6.6% of the throughput at the best α at every k measured.

Table 8: Throughput (QPS) at recall 0.90 and 0.95 on the five datasets in Figure 5, at $k = 10$ and $T = 64$, under both memory budgets. Bold indicates the highest QPS for each dataset, memory budget, and recall.

Budget	Dataset	Recall 0.90				Recall 0.95			
		RaBitQ-SSD	DiskANN	PipeANN	Starling	RaBitQ-SSD	DiskANN	PipeANN	Starling
$D_m = D$	OpenAI-5M	43 418	25 759	26 484	25 466	22 931	18 717	18 360	18 598
	Wiki-10M	42 279	36 847	31 455	35 965	28 571	25 209	20 778	24 181
	DPR-100M	6 690	2 946	3 834	3 333	2 829	1 291	1 736	1 512
	LAION-100M	50 509	39 366	40 233	36 109	32 252	27 175	27 207	25 740
	YFCC-100M	42 057	23 017	26 701	25 568	22 418	17 788	19 231	20 073
$D_m = D/2$	OpenAI-5M	32 818	16 695	16 561	16 590	24 345	10 234	9 700	10 147
	Wiki-10M	21 047	18 559	16 206	17 283	17 022	10 388	9 164	9 827
	DPR-100M	8 298	310	592	338	3 814	142	236	97
	LAION-100M	26 631	18 976	19 162	19 177	21 002	9 383	9 182	9 858
	YFCC-100M	36 465	14 851	16 360	16 367	26 120	9 427	9 961	10 349

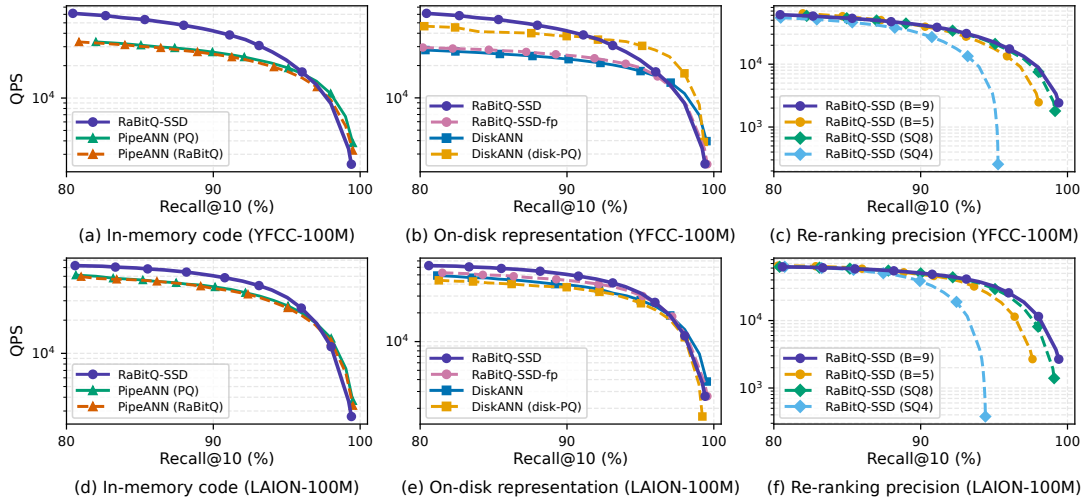


Figure 13: Ablation on YFCC-100M (top row) and LAION-100M (bottom row), at $D_m = D$, $k = 10$, and $T = 64$. The columns compare the in-memory code, on-disk representation, and re-ranking precision, respectively. All QPS axes use a log scale.

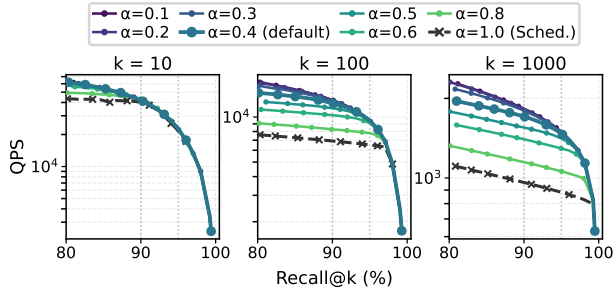


Figure 15: The drain threshold α on YFCC-100M at $T = 64$ and $D_m = D$: throughput against recall, one panel per k , one curve per α (log scale). $\alpha = 1.0$ disables the drain tightening and recovers Scheduled (§4.4); the heavier curve is the default $\alpha = 0.4$.

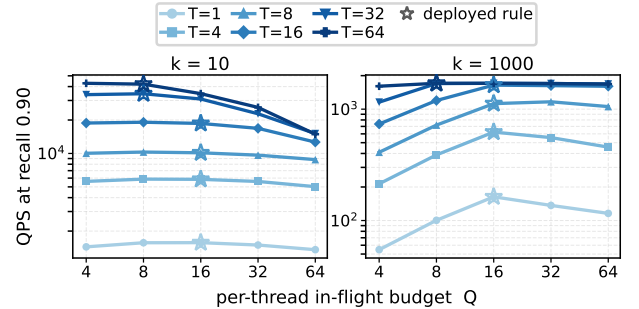


Figure 14: The in-flight budget Q on YFCC-100M ($D_m = D$): QPS at recall 0.90 against the per-thread Q , one line per thread count. Stars mark the deployed rule ($Q = 16$ for $T \leq 16$, 8 for $T > 16$).